

Grammar in Numbers

Arithmetic, Number Theory, and the Computational Grammar of Number

Abed Kadaan

Founding Principal Researcher

A frontier volume of the Computational Grammar Theory research program

Working edition 0.1 · October 2026

Research manuscript, not peer reviewed. Results marked “proved here” are self-checked and exhaustively verified on small cases by the accompanying code; nearly all are classical or elementary. Code, data and reproduction: github.com/akadaan310/grammarinnumbers

Contents

I. Charter and Provenance

Chapter 1. Research Charter: Grammar in Numbers

Chapter 2. Provenance Record

II. The Grammar of Number

Chapter 3. The Grammar of Number: 0, 1, 2, 3, 4, ...

Chapter 4. Definitions (provisional, session 1)

III. Arithmetic, Zero and Admissibility

Chapter 5. The Grammar of Arithmetic: +, −, ×, /

Chapter 6. Results Ledger

IV. Machines

Chapter 7. Machine Models: how a mathematical grammar becomes an executable one

Chapter 8. Formal Models

V. Number Theory

Chapter 9. Number Theory as Transition Systems

VI. Evidence

Chapter 10. Experiments

Chapter 11. Experimental summary (generated by `experiments/analyze.py`)

Chapter 12. Hypothesis Ledger

Chapter 13. Counterexamples and Negative Results

VII. CGT and the Frontier

Chapter 14. CGT Concept Map: how Computational Grammar Theory behaves on numbers

Chapter 15. Open Problems

Chapter 16. Literature Review (session 1 baseline)

Chapter 17. Book Outline

Chapter 18. Decision Log

Bibliography

I. Charter and Provenance

Research Charter: Grammar in Numbers

Founding Principal Researcher: Abed Kadaan **Parent research program:** Computational Grammar Theory (CGT), github.com/akadaan310/computationalgrammartheory **Research collaborator for this phase:** Claude (AI research assistant) **Charter date:** 2026-10-08 (session 1) **Status:** independent research publication, discovery phase. Nothing here is a finished theory.

1. The question

What is the computational grammar of number?

Grammar in Numbers (GiN) asks whether arithmetic and elementary number theory can themselves be described as computational grammars in the sense of CGT: a structure, a signature of moves, a relational semantics, an admissibility language and an explicit cost model. It does not begin by assuming that $1 + 1 = 2$ or $6 / 3 = 2$ are primitive notation that needs no structural investigation. It asks, for every arithmetic expression:

1. **Objects.** What are the values, and what are the symbols that name them?
2. **Moves.** What are the admissible operations, and what transitions do they induce?
3. **Admissibility.** When does an expression denote a result, when does it denote nothing, and when does it denote too much?
4. **Representation.** What changes when the same value is written in a different numeral system, stored in a different machine format, or computed by a different circuit?
5. **Cost.** What does each operation cost, in bit operations, under an explicit machine model?
6. **Novelty.** Which findings are new? Which are classical results restated, which are reinterpretations, which are measurements, which are conjectures?

The first target is not “advanced number theory”. It is **the grammar of number itself**: 0, 1, 2, 3, 4, ... and the operations +, −, ×, /.

2. Relationship to CGT

GiN is an *independent* publication. It is not a chapter of the CGT book.

- The CGT repository is a **read-only reference corpus**. GiN may read it, cite it, compare with it and propose extensions to it. GiN never modifies it. (See `PROVENANCE.md` §3 for the boundary and how it was enforced.)
- GiN is independently buildable: it has its own SDK (`sdk/`, package `ginsdk`), experiments, book sources and website build. It imports no CGT code at runtime.
- The relationship of every CGT concept to arithmetic is recorded in `CGT_CONCEPT_MAP.md`: which concepts survive unchanged, which need refinement, which become especially clear, and where arithmetic shows something CGT had not isolated.

3. Rules of the work

The CGT discipline is inherited, with additions specific to arithmetic.

- **Order.** Discover, then formalize, then experiment, then prove or falsify. The book and website come last.
- **Ladder of claims.** Every claim has exactly one category from this list:

Label	Meaning
DEFINITION	a precise object; not a discovery
AXIOM	an assumption of a formal system being studied
LEMMA / PROPOSITION / THEOREM / COROLLARY	proved statements (proof given here or cited)
HISTORICAL RESULT	a classical theorem, cited to its source
REINTERPRETATION	a known fact re-expressed in CGT/GiN terms; not a new result
CONJECTURE	believed, not established
EMPIRICAL OBSERVATION	a reproducible measurement; not a proof
IMPLEMENTATION RESULT	a property of code written here (e.g. “the simulator agrees with hardware on N cases”)
COUNTEREXAMPLE	an instance where a proposed claim fails
OPEN PROBLEM	a precisely stated unanswered question

Nothing is silently promoted. A visualization is not a proof. A benchmark is not a theorem. A new notation is not a new theory. A reinterpretation is not a new result. An implementation is not an algorithmic improvement.

- **Bit complexity is mandatory.** Unit-cost arithmetic (“one addition costs 1”) is used only when the operand size is bounded and the bound is stated. Otherwise costs are in bit operations or in w-bit word operations on a stated machine model (`FORMAL_MODELS.md` §1).

- **Number, numeral, encoding and machine state are kept apart** (DEFINITIONS.md GIN-DEF-001...004). Any claim that crosses layers says so.
- **“The machine” is never universal.** Every claim about machine behaviour names the ISA, the language, the compiler and the version, and says whether it was *measured on this host* or *taken from a specification* (MACHINE_MODEL.md).
- **“Undefined” is never the end of an analysis.** For every inadmissible expression the ledger states *which* condition fails (no solution, more than one solution, outside the domain, outside the representable range) and what would have to change for it to have a value.
- **Counterexamples are first-class results** (COUNTEREXAMPLES.md).
- **No priority claims without literature review.** Most of arithmetic is millennia old. The default assumption for any fact here is that it is known. Novelty is claimed only for precisely isolated statements, and even then marked “novelty unverified” until checked (LITERATURE_REVIEW.md, PROVENANCE.md).
- **Self-containment and reproducibility.** The SDK and experiments use the Python standard library only. Machine experiments that run compiled C, Java, Rust or JavaScript record the exact toolchain. Every experiment is deterministic given its seed.
- **History is preserved.** Rejected hypotheses stay in the ledger. Revisions go in DECISION_LOG.md.

4. Identifier convention

GIN-DEF-*nnn* definition · GIN-AX-*nnn* axiom system · GIN-H-*nnn* hypothesis · GIN-CONJ-*nnn* conjecture · GIN-LEM/PROP/THM/COR-*nnn* proved statements · GIN-HIST-*nnn* historical result · GIN-REI-*nnn* reinterpretation · GIN-OBS-*nnn* empirical observation · GIN-IMP-*nnn* implementation result · GIN-EXP-*nnn* experiment · GIN-NEG-*nnn* counterexample · GIN-REJ-*nnn* rejected hypothesis · GIN-OPEN-*nnn* open problem · GIN-D-*nnn* decision. CGT identifiers (CGT-*...*) always refer to the parent corpus.

5. Workspace

Path	Purpose
README.md	entry point and summary of findings
RESEARCH_CHARTER.md	this document
PROVENANCE.md	contributors, origin of every result, the CGT read-only boundary
CGT_CONCEPT_MAP.md	how each CGT concept behaves on numbers
DEFINITIONS.md	GIN-DEF-xxx
NUMBER_GRAMMAR.md	the grammar of 0, 1, 2, 3, ...: successor, numerals, addresses
ARITHMETIC_GRAMMAR.md	+, −, ×, / as transitions; converse problems; admissibility
FORMAL_MODELS.md	cost models, the four layers, the expression language
MACHINE_MODEL.md	ISA/language semantics, measured and specified
NUMBER_THEORY.md	divisibility, Euclid, congruences, primes as transition systems
HYPOTHESES.md	GIN-H-xxx with status
EXPERIMENTS.md	protocols GIN-EXP-xxx
RESULTS.md	propositions, theorems, observations, with proofs
COUNTEREXAMPLES.md	GIN-NEG-xxx, rejected hypotheses
LITERATURE_REVIEW.md	prior work by area
OPEN_PROBLEMS.md	GIN-OPEN-xxx
BOOK_OUTLINE.md	the book’s architecture, revised from the evidence
DECISION_LOG.md	dated decisions
sdk/	ginsdk, the executable companion (Python ≥ 3.10, stdlib only) with tests
experiments/	reproducible experiments, raw JSON, results/SUMMARY.md
book/	book sources (book/content), build tools, website assets, tests

6. Exit criteria for a first edition

1. The definitions of the four layers and of converse admissibility are stable across a second round of experiments.
2. Every “Proved here” statement is checked exhaustively on small cases by code in experiments/check_theorems.py.
3. Every machine claim is either measured on a recorded toolchain or cited to a specification.
4. The frontier assessment (RESULTS.md \$F) honestly states what, if anything, is new.

Provenance Record

1. Originator

Abed Kadaan is the Founding Principal Researcher of Grammar in Numbers (GiN). He formulated the research question (“What is the computational grammar of number?”), its relationship to Computational Grammar Theory, the central experimental narrative

$0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4$, then $1 + 1 \rightarrow 2$, $2 + 2 \rightarrow 4$, $0 / 1 \rightarrow 0$, $1 / 0 \rightarrow ?$, $0 / 0 \rightarrow ?$

the proposal to read $1 / 0$ and $0 / 0$ as two *different* grammatical failures (no candidate versus too many candidates), the segment notation $0\{1,2,3\}4$ as an object of investigation, the requirement that number, numeral, digit, machine word and abstract value be distinguished, and the governance rules of this publication (ladder of claims, read-only CGT boundary, honesty about novelty).

Kadaan is also the originator of the parent program, Computational Grammar Theory (CGT); see the CGT repository’s `PROVENANCE.md`.

2. Contributors in this phase

- **Abed Kadaan**: research program, research mandate, the questions listed above, governance rules.
- **Claude (AI research assistant, session 1, 2026-10-08)**: workspace, definitions GIN-DEF-xxx, hypotheses GIN-H-xxx, the `ginsdk` SDK, all experiment code, proofs written in `RESULTS.md`, literature baseline, book text and website. All of it is subject to review; nothing has been externally reviewed.

3. The CGT boundary (read-only)

The CGT repository (`akadaan310/computationalgrammartheory`, branch `claude/happy-newton-r2t17c`, head `b3a29f3` at the time of reading) was **read, never modified**.

- What was read: all ledger files (`DEFINITIONS.md`, `FORMAL_MODELS.md`, `RESULTS.md`, `HYPOTHESES.md`, `COUNTEREXAMPLES.md`, `OPEN_PROBLEMS.md`, `PROVENANCE.md`, `DECISION_LOG.md`, `BOOK_OUTLINE.md`, `PUBLICATION_AUDIT.md`, `EXPERIMENTS.md`), the book sources’ structure, the book build tool `book/tools/build.mjs`, its example checker, its stylesheet and its JavaScript.
- What was reused, and how: GiN’s book build (`book/tools/build.mjs`) and example checker follow the *design* of CGT’s (Markdown + KaTeX at build time, environments with status badges, build-time integrity checks). The GiN files were written for this repository and are adapted to its content; there is no runtime or build-time dependency on the CGT repository. The visual identity deliberately shares CGT’s typography and paper palette and changes the accent colour, so that the two publications read as one family.
- What was *not* reused: no CGT code is imported by `ginsdk` or by any experiment. Where a GiN result concerns a CGT object (for example the heap numeration of CGT-DEF-006), GiN re-implements it independently (`ginsdk.numerals.heap_word`).
- Verification: `git status` in the CGT checkout was clean at the end of the session (`DECISION_LOG.md`, GIN-D-002).

CGT identifiers cited in GiN (CGT-DEF-006, CGT-THM-003, CGT-THM-004, CGT-THM-006, CGT-DEF-011, CGT-PROP-002, ...) refer to the CGT ledger at the commit above.

4. Classification of every result

“Proved here” means a proof is written in `RESULTS.md` by the session-1 collaborator and checked exhaustively on small cases by `experiments/check_theorems.py`; it does **not** mean the statement is new. Almost everything in arithmetic is old.

ID	Category	Origin	Relationship to prior work
GIN-DEF-001...004 (four layers)	definition	this investigation, from Kadaan’s mandate	standard distinctions (number/numeral; value/representation) made explicit as layers
GIN-DEF-005...008 digit grammars, canonical numerals	definition	this investigation	positional notation is classical; the CGT packaging (moves, address, transition monoid) is new framing
GIN-DEF-012 segment notation $a\{I\}b$	definition	Kadaan’s proposal, formalized here	equivalent to a marked interval; no new mathematics (see <code>RESULTS</code> §B)
GIN-DEF-015...017 converse problem, admissibility, repairs	definition	this investigation	relational converse is standard; the classification vocabulary is GiN’s
GIN-PROP-001...005, 008, 009	proposition	classical facts, proofs written here	positional numeration (Knuth TAOCP vol. 2 §4.1); bijective numeration (Smullyan 1961; Foster 1947 [U])
GIN-PROP-006 heap conjugacy	proposition	proved here	elementary; very likely folklore; novelty unverified
GIN-PROP-010 increment flips $2N - \text{popcount}(N)$	proposition	proved here	the amortized binary counter is textbook (CLRS §17); the exact closed form is elementary and almost certainly known
GIN-THM-001...003 converse structure in rings	theorem	standard algebra, proofs written here	elementary ring theory
GIN-THM-004 extension trilemma	reinterpretation	this investigation	synthesizes cited facts about extended reals, projective line, IEEE 754, meadows, wheels

ID	Category	Origin	Relationship to prior work
GIN-THM-005 addition finite-state, multiplication not	theorem	known (Büchi; Hodgson; Khossainov & Nerode; Blumensath & Grädel; Gödel/Church)	restated as a grammar-class boundary
GIN-PROP-020 carry monoid	proposition	known in substance (Brent & Kung 1982; Ladner & Fischer 1980)	restated as CGT arithmetic compilation
GIN-PROP-021 ripple counts	proposition	proved here	elementary, for the specific construction
GIN-PROP-023 restoring divider on 0	proposition	proved here	the RISC-V specification’s own rationale already says all-ones is “the natural value” for simple dividers; GiN supplies a proof and gate-level check, not a new fact
GIN-PROP-030, 031 representability	proposition	proved here	elementary and well known
GIN-PROP-040...043 number theory	historical / proposition	classical (Euclid; Lamé 1844; Bézout; Sunzi/CRT; Fermat; Korselt 1899; Miller 1976; Rabin 1980)	restated as transition systems
GIN-HIST-001 Brahmagupta’s zero rules	historical result	Brahmagupta 628 CE, via secondary sources	GiN’s reading of $0/0 = 0$ as a <i>selection</i> (a totalization), not an error of derivation, is an interpretation
GIN-OBS-xxx	empirical	experiments here	n/a
GIN-IMP-001 IEEE simulator agrees with hardware	implementation result	experiments here	the method (exact rational then one rounding) is the standard’s own definition
GIN-NEG-xxx	counterexamples	experiments and analysis here	several are well known (float non-associativity, pseudoprimes)

5. Priority statement

No claim of historical priority is made for any mathematical result in this repository. The candidate contributions of session 1 are: (i) a **framing** of arithmetic as CGT operational grammars with an explicit four-layer outcome model (syntax · typing · semantics · execution); (ii) the **converse-admissibility** reading of inverse operations, which classifies $1 / 0$ (no solution) and $0 / 0$ (non-unique) uniformly with $0 - 1$ in $\mathbb{N}$, $\sqrt{-1}$ in $\mathbb{R}$, $7 / 2$ in $\mathbb{Z}$ and modular inverses; (iii) a set of **executable, cross-validated artefacts** (an IEEE simulator matched to hardware including flags, measured machine semantics across seven toolchains, gate-level circuits); (iv) several **precise links to CGT** (numerals as addresses, carry-lookahead as arithmetic compilation, Horner vs. balanced evaluation as a grammar-shape trade-off). Whether any of this is new beyond presentation is assessed honestly in `RESULTS.md` §F.

II. The Grammar of Number

The Grammar of Number: 0, 1, 2, 3, 4, ...

This document develops the first target of the research: the grammar of number itself. Formal statements are in `DEFINITIONS.md` and `RESULTS.md`; this is the synthesis.

1. Twelve readings of “3”, kept apart

Reading	What “3” is	Layer (GIN-DEF-001...004)
mathematical object	the element $S(S(0))$ of the successor algebra	value
count	the cardinality of $\{a, b, c\}$	value (via cardinality)
position	the fourth place in 0, 1, 2, 3, ... (or the third, counting from 1)	value with an order convention
state	a node of the successor transition system T_N	value as state
address	the canonical numeral $\text{addr}(3)$ in some digit grammar: 3, 11 ₂ , 55 ₅ , 10 ₃	numeral
digit	the symbol 3 of the decimal alphabet	numeral alphabet
symbol	a token in an expression such as $3 + 1$	syntax
sequence	the word 3 in the language of decimal numerals	numeral
machine representation	0x00000003 (int32), 0x4008000000000000 (binary64)	encoding
transition result	the value of $1 + 2$, $S(2)$, $6 / 2$	value (output of a move)
algebraic element	$3 \in \mathbb{Z}, \mathbb{Q}, \mathbb{R}, \mathbb{C}$; the class $3 + n\mathbb{Z}$ in $\mathbb{Z}/n$; $3 = 0$ in $\mathbb{Z}/3$	value in a chosen structure
computational value	what a register holds after <code>mov eax, 3</code>	machine state

Where they agree: every reading denotes, or encodes, the same value 3 in $\mathbb{N}$. Where they diverge: a digit is not a number (3 is a symbol); a numeral is not a number (03 and 3 are different words); a binary representation is not the number (11₂ vs 3); a machine word is not the integer (0xFFFFFFFF decodes to -1 or 4,294,967,295 depending on signedness); a residue class is not an integer ($3 = 0$ in $\mathbb{Z}/3$).

2. The smallest grammar: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4$

The structural grammar $N \rightarrow 0 \mid S(N)$ generates the terms 0, $S(0)$, $S(S(0))$, Read as a transition system T_N , the transition $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4$ is four applications of the single move S . Three facts make this the smallest laboratory for transition grammar:

- Every value is reached from 0 by a unique word** (S^n), so the unary numeral is simultaneously the value’s *name*, its *address* and the *program* that computes it.
- The converse move P is partial**: P is inadmissible at 0. This is the first boundary of arithmetic, and it is the boundary that $0 - 1$ meets in $\mathbb{N}$.
- $n + 1$ is a grammatical transition, not merely notation**: by the primitive-recursive definition $\text{add}(m, S n) = S(\text{add}(m, n))$, adding 1 *is* one application of S , and adding k is k applications (GIN-PROP-003).

The segment $[0, 4]$ — written 0{1,2,3}4 in the notation proposed by A. Kadaan (GIN-DEF-012) — is the finite neighbourhood in which these facts can be inspected: the boundary $\{0, 4\}$ where exactly one move is admissible, the interior $\{1, 2, 3\}$ where both are. GIN-REI-010 records the honest verdict: the notation is formal (checkable, diagnosable) but adds no mathematics.

3. Numerals as programs

A positional numeral is a word over digit moves $d : x \mapsto b \cdot x + d$ (GIN-DEF-005). Consequences, each proved in `RESULTS.md`:

- Numeral $\neq$ number** (PROP-001): the value map is surjective and non-injective exactly by leading zeros.
- Zero’s address is the empty word** (PROP-002): the written “0” is a convention.
- Bijective notation** has no zero digit and is a bijection between words and $\mathbb{N}$ (PROP-004).
- Compilation**: a numeral compiles to the affine map $x \mapsto b^{|w|}x + \text{val}(w)$ (PROP-005); concatenation of numerals is composition of maps.
- CGT’s heap numeration is bijective binary shifted by one** (PROP-006).
- Positional notation is compression**: the binary numeral of n is a straight-line program of size $O(\log n)$ for the unary term $S^n(0)$ — CGT’s M-SHARE — with exactly the shape of square-and-multiply exponentiation (PROP-007, PROP-041).
- Negative numbers have no finite standard numerals**: the residue expansion of -1 is ...111, the 2-adic expansion; fixed-width two’s complement keeps its first w digits (PROP-008). Balanced ternary names every integer finitely (PROP-004).
- Rationals have eventually periodic expansions** produced by a finite transition system on remainders (PROP-009).

4. Number systems as grammars

Carrier	Total ops	Admissibility of inverses	Identities	Order	Typical representations
$\mathbb{N}$	$+, \times$	$a - b$ iff $a \geq b$; a/b iff $b \neq 0$ and $b \mid a$	0 (+), 1 ($\times$)	total, well-founded	unary, positional, bijective
$\mathbb{Z}$	$+, -, \times$	a/b iff $b \neq 0$ and $b \mid a$	0, 1	total	sign-magnitude, balanced ternary, two’s complement (fixed width)

Carrier	Total ops	Admissibility of inverses	Identities	Order	Typical representations
$\mathbb{Q}$	$+, -, \times$	a/b iff $b \neq 0$	0, 1	total, dense	pairs in lowest terms; periodic positional expansions
$\mathbb{R}$	$+, -, \times$	a/b iff $b \neq 0$; $\sqrt{a}$ iff $a \geq 0$ (two roots, principal branch)	0, 1	total, complete	infinite expansions; computable reals; floating-point approximations
$\mathbb{C}$	$+, -, \times$	a/b iff $b \neq 0$; $\sqrt{a}$ always (two roots for $a \neq 0$); $\log a$ iff $a \neq 0$ (infinitely many)	0, 1	none compatible with the field	pairs of reals
$\mathbb{Z}/n$	$+, -, \times$	a/b iff $\gcd(b, n) \mid a$, unique iff $\gcd(b, n) = 1$ (THM-002)	0, 1	none compatible	residues $0 \dots n-1$
$\mathbb{Z}_p$ (p-adic integers)	$+, -, \times$	a/b iff $v_p(b) \leq v_p(a)$	0, 1	none compatible	infinite digit sequences to the left
w-bit words	$+, -, \times \bmod 2^w$	division per machine model (MACHINE_MODEL)	0, 1	signed/unsigned interpretations	bit strings
IEEE binary64	$+, -, \times, \div, \sqrt{}$ (all total, with special data)	no inverse is exact in general; $x/0 = \pm\infty$, $0/0 = \text{NaN}$	+0 for + (with signed-zero rules), 1 for $\times$	total on non-NaN (with $-0 = +0$)	64-bit strings

Each step $\mathbb{N} \rightarrow \mathbb{Z} \rightarrow \mathbb{Q} \rightarrow \mathbb{R} \rightarrow \mathbb{C}$ is an admissibility repair (GIN-DEF-017, GIN-PROP-017): the carrier grows exactly so that a family of converse problems acquires solutions. $\mathbb{Z}/n$ and fixed-width words take the other route — a quotient that makes subtraction total by giving up order. Floating point takes a third route: it keeps every operation total by giving up exactness and some algebraic laws (associativity, PROP-032; the converse reading of division, PROP-015).

5. Zero and one

Zero is: the additive identity; the empty count ($|\emptyset|$); the base of the successor grammar; the address-free value (empty numeral); a positional placeholder; an absorbing element for $\times$ (derived in rings, assumed in semirings: PROP-012); the boundary of P in T_N ; the fixed point of $x \mapsto 2x$ and of $x \mapsto x^2$; the origin; and, in machines, the bit pattern `0...0`. Zero is **not**: NULL (absence of a reference), undefined (no meaning), the empty string (a numeral, not a number), false (a truth value, encoded identically in many languages), absence, uninitialized memory (no defined value; reading it can be undefined behaviour in C), or NaN (a datum that is not a number and is unequal to itself). IEEE 754 has two zeros, $+0$ and -0 , equal under comparison but distinguished by $1/x$.

One is: the multiplicative identity; the unit; $S(0)$; the generator of $\mathbb{N}$ under $+$ (every n is $1 + 1 + \dots + 1$); the scale of positional notation's lowest place; a fixed point of squaring. The distinction **identity vs generator** matters: 0 is the identity of $+$ but generates nothing; 1 is the identity of $\times$ but generates all of $\mathbb{N}$ under $+$. The transitions $0 \rightarrow 1$ and $1 \rightarrow 2$ are the same move S , but $0 \rightarrow 1$ crosses from the empty count to the first non-empty one, and only 0 has no predecessor.

6. Fixed points

Sense	Example	Computational content
$f(x) = x$	$x^2 = x$ has solutions 0, 1 in $\mathbb{Z}$; four idempotents in $\mathbb{Z}/6$ (0, 1, 3, 4)	solving an equation (a converse problem)
limit of iteration	Newton's $x \mapsto (x + a/x)/2$ converges to $\sqrt{a}$; CGT's PageRank	existence, uniqueness, rate, stopping rule
least fixed point	$\mathbb{N} = \text{least } X \text{ with } \{0\} \cup S(X) \subseteq X$	induction, recursion; the definition of the grammar itself
terminal state	Euclid's $(g, 0)$; a state with no admissible move	termination of a transition system

These share a definition, not a method (GIN-REI-007).

Definitions (provisional, session 1)

Each definition is marked *[standard]* (an established notion restated), *[GiN, provisional]* (introduced by this investigation) or *[GiN naming of standard notion]*. Existing terms are not redefined when an existing term does the job. CGT identifiers refer to the parent corpus (read-only).

A. The four layers

GIN-DEF-001 Number (value) *[standard]*

A **natural number** is an element of a *successor algebra* $(\mathbb{N}, 0, S)$: a set with a distinguished element 0 and an injective map S with $0 \notin S(\mathbb{N})$, such that every subset containing 0 and closed under S is all of $\mathbb{N}$ (Dedekind 1888; Peano 1889). Any two such algebras are uniquely isomorphic, so “the” natural numbers are well defined up to unique isomorphism. Integers, rationals, reals and complex numbers are defined from $\mathbb{N}$ by standard constructions (GIN-DEF-020). A *value* is an element of one of these structures. Values are not strings and have no digits.

GIN-DEF-002 Numeral *[standard; GiN packaging]*

A **numeral** is a word over a finite digit alphabet, read in a *numeral system* that assigns it a value (GIN-DEF-005). 2, 10₂, II, 55, 0010 are numerals. A **digit** is a letter of a numeral alphabet; a digit is a symbol, not a number, even when a numeral of length one denotes the same value as its only digit.

GIN-DEF-003 Encoding *[standard]*

An **encoding** of a set V of values in a format F is a partial injective map $\text{enc}: V \rightarrow \{0,1\}^w$ together with a total decoding map dec from bit strings to $V \cup S_F$, where S_F is a set of *special data* that are not values (for IEEE 754: $\pm\infty$, NaN; the sign of zero is extra information). Examples: w -bit two’s complement encodes $\{-2^{\wedge}\{w-1\}, 2^{\wedge}\{w-1\}\}$; IEEE binary64 encodes a finite set of dyadic rationals.

GIN-DEF-004 Machine state *[standard]*

A **machine state** is an assignment of bit strings to registers and memory together with status information (flags, pending exceptions, the trap state). An **instruction** is a partial map on states. The *execution* of an expression is a sequence of instructions; its *outcome* is either a final state from which a value is decoded, or a trap/exception state.

The four layers. value (DEF-001) · numeral (DEF-002) · encoding (DEF-003) · machine state (DEF-004). Maps between them: denotation (numeral $\rightarrow$ value), encoding (value $\rightarrow$ bits), decoding (bits $\rightarrow$ value or special datum), execution (state $\rightarrow$ state). A claim about arithmetic names its layer. “1 + 1 = 2” is a statement about values; “1 + 1 = 10₂” is the same statement with the right-hand side written in a different numeral system; “0x7FFFFFFF + 1 is 0x80000000” is a statement about encodings under wraparound.

B. Numerals as operation expressions

GIN-DEF-005 Digit grammar *[GiN, provisional; positional notation is classical]*

A **digit grammar** is a pair (b, D) with base $b \geq 1$ and a finite digit set $D \subset \mathbb{Z}$. It is a CGT operational grammar (CGT-DEF-003) on the carrier $\mathbb{Z}$ with one move per digit,

$$\llbracket d \rrbracket : x \mapsto b \cdot x + d \quad (d \in D),$$

start point 0, and admissibility language D^* (or a sublanguage, GIN-DEF-007). Instances: *standard base* b ($b \geq 2$, $D = \{0, \dots, b-1\}$); *bijective base* b ($D = \{1, \dots, b\}$; $b = 1$ is unary, whose single move is the successor S); *balanced ternary* ($b = 3$, $D = \{-1, 0, 1\}$); redundant systems (e.g. $b = 2$, $D = \{0, 1, 2\}$).

GIN-DEF-006 Value of a numeral *[standard]*

The **value** of a word $w = d_1 d_2 \dots d_k$ (most significant digit first) is $\text{val}(w) = \llbracket w \rrbracket(0) = \sum_i d_i b^{\wedge}\{k-i\}$. Evaluating $\llbracket w \rrbracket$ left to right is Horner’s rule.

GIN-DEF-007 Canonical numeral (address) *[GiN naming of standard notion; specialization of CGT-DEF-006]*

The **canonical numeral** $\text{addr}(n)$ of a value n is the shortlex-least word w with $\llbracket w \rrbracket(0) = n$, when one exists. For standard base b it is the usual numeral without leading zeros, and $\text{addr}(0) = \epsilon$, the empty word.

GIN-DEF-008 Affine transition monoid *[standard; CGT-DEF-007 instance]*

The transition monoid of a digit grammar is the set of maps $\llbracket w \rrbracket = (x \mapsto b^{|w|}x + \text{val}(w))$, $w \in D^*$, under composition. A word **compiles** to the pair $(b^{|w|}, \text{val}(w))$.

C. The successor grammar

GIN-DEF-009 Successor grammar [standard]

The structural grammar $\mathbb{N} \rightarrow 0 \mid S(\mathbb{N})$ generates the *successor terms* $0, S(0), S(S(0)), \dots$; its unique model is the successor algebra (GIN-DEF-001). A successor term is a unary numeral (bijective base 1).

GIN-DEF-010 Primitive recursion on the successor grammar [standard]

$\text{add}(m, 0) = m$, $\text{add}(m, S n) = S(\text{add}(m, n))$; $\text{mul}(m, 0) = 0$, $\text{mul}(m, S n) = \text{add}(\text{mul}(m, n), m)$; $\text{pow}(m, 0) = S 0$, $\text{pow}(m, S n) = \text{mul}(\text{pow}(m, n), m)$; $\text{pred}(0) = 0$, $\text{pred}(S n) = n$; $\text{monus}(m, 0) = m$, $\text{monus}(m, S n) = \text{pred}(\text{monus}(m, n))$. Cost is counted in **rule applications** (one per equation used).

GIN-DEF-011 Successor transition system [GiN naming of standard notion]

$T_{\mathbb{N}}$ is the labelled transition system on $\mathbb{N}$ with move S (total) and its converse P (defined on $\mathbb{N} \setminus \{0\}$). The **boundary** of a subset $X \subseteq \mathbb{N}$ in $T_{\mathbb{N}}$ is the set of $x \in X$ at which S or P leads out of X or is undefined.

GIN-DEF-012 Segment and the notation $\mathbf{a}\{i\}b$ [GiN, provisional; notation proposed by A. Kadaan]

For integers $a < b$, the **segment** $[a, b]$ of $T_{\mathbb{N}}$ (or $T_{\mathbb{Z}}$) is the transition system on $\{a, \dots, b\}$ with S defined on $a \dots b-1$ and P on $a+1 \dots b$. Its *boundary* is $\{a, b\}$ (each has exactly one admissible move) and its *interior* is $\{a+1, \dots, b-1\}$ (both moves admissible). The notation $\mathbf{a}\{i_1, \dots, i_k\}b$ is *well formed* iff $a < b$ and $(i_1, \dots, i_k) = (a+1, \dots, b-1)$; it then denotes the segment $[a, b]$ with its boundary written outside the braces and its interior inside. Thus $\emptyset\{1,2,3\}4$ denotes $[0, 4]$; $3\{\}4$ denotes the segment with empty interior; $\emptyset\{1,3\}4$ and $4\{\}4$ are ill formed.

GIN-DEF-013 Church numeral [standard]

The Church numeral of n is the higher-order function $f \mapsto f^n$ (Church 1941): a number *as* an iterator. It is the successor grammar read as a program.

D. Operations and admissibility

GIN-DEF-014 Operation grammar [GiN, provisional]

A binary operation $\circ$ on a carrier A induces the relation $\llbracket \circ \rrbracket \subseteq (A \times A) \times A$, $\{((a, b), a \circ b)\}$, and, for each b , the move $\llbracket \circ b \rrbracket : a \mapsto a \circ b$. An operation is **closed** (total) on A if $a \circ b \in A$ for all $a, b \in A$.

GIN-DEF-015 Converse problem [GiN, provisional; relational converse is standard]

For a forward operation $\circ$ the **converse problem** at (a, b) is

$$\text{Sol}_{\circ}(a, b) = \{ c \in A : b \circ c = a \}.$$

Division asks $\text{Sol}_{\times}(a, b)$; subtraction asks $\text{Sol}_{+}(a, b)$; the square root asks $\text{Sol}(a) = \{c : c \cdot c = a\}$; the logarithm asks $\{c : e^c = a\}$; the modular inverse asks $\text{Sol}_{\times}(1, b)$ in $\mathbb{Z}/n$.

GIN-DEF-016 Converse admissibility [GiN, provisional]

An inverse expression is

- **inadmissible: no solution** if $\text{Sol} = \emptyset$;
- **admissible** if $|\text{Sol}| = 1$, and then denotes the unique solution;
- **inadmissible: non-unique** if $|\text{Sol}| \geq 2$.

A **branch** (principal value, selection) is a choice function on non-unique solution sets that turns a non-unique converse into a function (e.g. $\sqrt{}$ selects the nonnegative root).

GIN-DEF-017 Repairs of inadmissibility [GiN, provisional classification]

Ways in which a mathematical or computational system gives an inadmissible expression a value:

1. **Extension** — enlarge the carrier so that Sol becomes non-empty ($\mathbb{N} \rightarrow \mathbb{Z}$ for subtraction, $\mathbb{Z} \rightarrow \mathbb{Q}$ for division by nonzero, $\mathbb{R} \rightarrow \mathbb{C}$ for $\sqrt{}$ of negatives, $\mathbb{R} \rightarrow \mathbb{R} \cup \{\infty\}$ for $1/0$).
2. **Selection** — choose an element of a non-unique Sol (principal branches; Euclidean quotient with remainder conditions).
3. **Restriction** — remove the bad inputs from the domain ($b \neq 0$; units only).
4. **Quotient** — identify elements so that the operation becomes total ($\mathbb{Z} \rightarrow \mathbb{Z}/2^w$ makes $0 - 1 = 2^w - 1$).

5. **Totalization** — define a value by convention without claiming it solves the converse problem (meadows and proof assistants: $x/0 := 0$; AArch64 integer division by zero returns 0; RISC-V returns -1).
6. **Absorption** — adjoin a special element that absorbs further operations (NaN; the wheel element $\perp$; Option/Result types).
7. **Signal** — refuse to produce a value and transfer control (hardware trap, exception, panic, compile-time error).

Each repair changes the grammar; GIN-THM-004 records *which* law each one gives up.

GIN-DEF-018 Closure of a domain *[standard]*

A domain is **closed** under an inverse operation if the operation is admissible at every point. $\mathbb{Z}$ is closed under subtraction; $\mathbb{Q} \setminus \{0\}$ under division; $\mathbb{Z}/p$ (p prime) under division by nonzero elements.

GIN-DEF-019 Admissibility closure *[GiN, provisional]*

Given a carrier $A \subseteq U$ inside a larger structure U and a set of converse problems, the **admissibility closure** of A is the least B with $A \subseteq B \subseteq U$ such that every converse problem with data in B that has a solution in U has one in B . Examples: the closure of $\mathbb{N}$ in $\mathbb{Q}$ under subtraction is $\mathbb{Z}$; the closure of $\mathbb{Z}$ in $\mathbb{C}$ under division by nonzero elements is $\mathbb{Q}$; the closure of $\mathbb{Q}$ in $\mathbb{R}$ under field operations and square roots of positive elements is the field of real constructible numbers (see GIN-OPEN-010).

GIN-DEF-020 Number system as grammar *[GiN packaging of standard notions]*

A number system is presented as a tuple (carrier, operations, admissibility of each inverse, identities, order, topology where relevant, representations). `NUMBER_GRAMMAR.md` §4 gives the table for $\mathbb{N}$, $\mathbb{Z}$, $\mathbb{Q}$, $\mathbb{R}$, $\mathbb{C}$, $\mathbb{Z}/n$, $\mathbb{Z}_p$, fixed-width words and IEEE formats.

E. Floating point and machine models

GIN-DEF-030 IEEE binary format *[standard: IEEE 754-2019]*

A format with precision p (significand bits including the hidden bit) and maximum exponent $e_{\max} = 2^{\{e-1\}} - 1$ for e exponent bits; $e_{\min} = 1 - e_{\max}$. Finite data are $\pm m \cdot 2^{\{q\}}$ with integers $0 \leq m < 2^p$ and $q \geq e_{\min} - p + 1$, plus $\pm\infty$ and NaN.

GIN-DEF-031 Correct rounding *[standard]*

An operation is **correctly rounded** if its result is the exact real result rounded once to the format (here: to nearest, ties to even). IEEE 754 requires this of $+$, $-$, $\times$, $\div$, $\sqrt{}$ and fused multiply-add.

GIN-DEF-032 Exception flags *[standard]*

invalid (no useful real result: $0/0$, $\infty - \infty$, $0 \times \infty$, $\sqrt{\text{negative}}$), operations on signaling NaNs), divideByZero (exact infinite result from finite operands: $x/0$ with $x \neq 0$ finite), overflow, underflow (tiny and inexact under default handling), inexact.

GIN-DEF-033 Integer machine model *[GiN packaging]*

A triple (width w , overflow policy $\in \{\text{wrap, saturate, trap, undefined}\}$, division semantics) where division semantics fixes the results or signals for $x/0$ and for $-2^{\{w-1\}}/(-1)$. `MACHINE_MODEL.md` lists the models for x86-64, AArch64, RISC-V, ISO C, Java, Rust, Go, Python and JavaScript.

F. Circuits

GIN-DEF-040 Circuit *[standard]*

A directed acyclic graph whose sources are input wires and constants and whose internal nodes are gates from a basis (here $\{\text{NOT, AND, OR, XOR}\}$ with fan-in ≤ 2 , or $\{\text{NAND}\}$). *Admissibility* is syntactic: each gate input is a previously defined wire. *Semantics*: the Boolean function computed. *Cost*: (size = number of gates, depth = longest input-output path).

GIN-DEF-041 Carry automaton and carry monoid *[standard idea; GiN naming]*

Binary addition read least significant digit first is a two-state transducer whose state is the carry. On a digit pair (a, b) its transition is a map on $\{0, 1\}$: **K** (kill, $c \mapsto 0$) for $(0, 0)$, **P** (propagate, $c \mapsto c$) for $(0, 1)$ and $(1, 0)$, **G** (generate, $c \mapsto 1$) for $(1, 1)$. The **carry monoid** is $\{K, P, G\}$ under composition.

GIN-DEF-042 Prefix adder *[standard]*

An adder that computes all carries as prefix compositions in the carry monoid with a parallel-prefix network (Sklansky 1960; Kogge & Stone 1973; Ladner & Fischer 1980; Brent & Kung 1982).

G. Number theory as transition systems

GIN-DEF-050 Euclid system *[standard]*

States $(a, b) \in \mathbb{N}^2$; move $(a, b) \rightarrow (b, a \bmod b)$ admissible iff $b \neq 0$; invariant $\gcd(a, b)$; terminal states $(g, 0)$.

GIN-DEF-051 Bézout rows *[standard]*

Rows (r_i, s_i, t_i) with $r_0 = a$, $r_1 = b$, $r_{i+1} = r_{i-1} - q_i r_i$ (same for s, t), invariant $r_i = s_i a + t_i b$.

GIN-DEF-052 Exponent program *[standard; GiN naming]*

For $e \geq 1$ with binary numeral 1β , the **exponent program** is the word over $\{M, S, SM\}$ that starts with M (load the base) and maps each later bit $0 \mapsto S$ (square), $1 \mapsto SM$ (square, then multiply). Running it computes x^e (left-to-right binary exponentiation).

GIN-DEF-053 Congruence system *[standard]*

A system $x \equiv r_i \pmod{m_i}$ is a converse problem for the map $x \mapsto (x \bmod m_1, \dots, x \bmod m_k)$; its solution set is $\emptyset$ or one residue class mod $\text{lcm}(m_i)$.

H. Arithmetic as a language

GIN-DEF-060 Arithmetic expression language *[GiN, provisional]*

Tokens: numerals (decimal integers and decimal fractions; `0b...` and `...2` binary; `0x...` hexadecimal), the variable x , operators $+ - \times //$ mod $^$, parentheses, functions `sqrt` and `gcd`. Grammar (precedence low to high): additive, multiplicative, unary minus, power (right-associative). Implemented in `ginsdk.expr`.

GIN-DEF-061 Four evaluation layers *[GiN, provisional; refines CGT's three layers]*

1. **syntax** — is the string a sentence of the grammar?
2. **typing** — does every literal and variable denote an element of the chosen domain?
3. **semantics** — does every operation have exactly one result (GIN-DEF-016)?
4. **execution** — what does the chosen machine model do (overflow, rounding, traps, exceptions, undefined behaviour, flags)?

GIN-DEF-062 Outcome *[GiN, provisional]*

The result of evaluating an expression in a domain: a status (value, special datum, no-solution, non-unique, not-in-domain, unsupported, overflow, trap, exception, undefined, syntax-error), the layer at which evaluation stopped, a reason, the converse equation where relevant, flags, the step trace, the parse tree, the representation of the result, and a cost estimate. No outcome is a bare “error”.

I. Cost

GIN-DEF-070 Bit-cost model *[standard]*

Operands are measured by bit length L . A *word operation* acts on w -bit limbs (here $w = 32$): add/subtract-with-carry, limb product, two-by-one-limb quotient estimate, limb comparison. Unit-cost arithmetic is used only for operands of bounded length, and the bound is stated.

III. Arithmetic, Zero and Admissibility

The Grammar of Arithmetic: +, −, ×, /

For each operation: input grammar, output grammar, admissibility, identity, composition, inverse, closure, machine implementation, cost, failure states, representation dependence. Proofs and evidence are cited from `RESULTS.md` and `EXPERIMENTS.md`.

1. The operation table

	+	−	×	/
input grammar	pairs of values	pairs	pairs	pairs
output	value	value or failure	value	value or failure
admissibility in $\mathbb{N}$	always	$a \geq b$	always	$b \neq 0$ and $b \mid a$
admissibility in a field	always	always	always	$b \neq 0$
identity	0	right identity 0 ($a - 0 = a$); no left identity	1	right identity 1
composition	associative, commutative	neither	associative, commutative	neither
inverse	− (converse)	+	/ (converse)	×
closure	$\mathbb{N}$	$\mathbb{Z}$	$\mathbb{N}$	$\mathbb{Q} \setminus \{0\}$ for nonzero divisors
definition as grammar	primitive recursion on S	converse of +	primitive recursion on +	converse of ×
bit cost (L-bit operands)	$\Theta(L)$	$\Theta(L)$	$\Theta(L^2)$ schoolbook, $\Theta(L^{1.585})$ Karatsuba, $O(L \log L)$ best known	$\Theta(L^2)$ schoolbook
failure states	overflow (fixed width)	no solution (N); overflow	overflow	no solution (image failure), non-unique (kernel failure), overflow ($\text{INT_MIN} - 1$)
representation dependence	carry pattern depends on base	borrow pattern	partial products	quotient digits; termination of expansion

2. Addition: $1 + 1 = 2$, five ways

1. **Counting:** $|\{a\}| + |\{b\}| = |\{a, b\}|$ for disjoint singletons.
2. **Successor:** $\text{add}(1, S\ 0) = S(\text{add}(1, 0)) = S(1) = 2$ — two rule applications.
3. **Cardinality:** + is the cardinality of a disjoint union; well defined because bijections compose.
4. **Algebra:** $1 + 1$ is the element 2 of the ring generated by 1.
5. **Machine:** binary $1 + 1 = 10_2$: the digit pair (1, 1) is a *generate* (G) in the carry monoid; the sum bit is 0 and the carry 1 moves one place left (GIN-DEF-041). In a half adder: $\text{sum} = \text{XOR}(1, 1) = 0$, $\text{carry} = \text{AND}(1, 1) = 1$.

Invariant: the value 2. *Generated:* a new position (the carry). *Merely represented:* the digit string 10 in base 2, 2 in base 10, 55 in unary. $1 + 2 = 3$, $2 + 2 = 4$, $3 + 1 = 4$, $4 + 1 = 5$ behave the same way; in binary, $3 + 1 = 11_2 + 1_2 = 100_2$ propagates a carry through two positions (one G, one P): the successor's worst case (GIN-PROP-010).

Finite-width boundary. In n -bit two's complement, $\text{MAX} + 1 = \text{MIN}$ under wraparound, saturates, traps, or is undefined behaviour in C (GIN-EXP-012). Arbitrary precision moves the boundary to memory and time.

3. Subtraction: $0 - 1$

Subtraction is the converse of addition: $a - b = c \Leftrightarrow b + c = a$. In $\mathbb{N}$, $\text{Sol}(0, 1) = \{c : 1 + c = 0\} = \emptyset$ — no solution. In $\mathbb{Z}$ it is $\{-1\}$.

What changed? The *structure* was extended ($\mathbb{N} \rightarrow \mathbb{Z}$, the Grothendieck construction), which changed the admissible-result grammar; the *grammar of the operation* (converse of +) did not change. In $\mathbb{Z}/2^w$ (hardware) the answer is $2^w - 1$: a quotient, not an extension — order is given up. Truncated subtraction $0 - 1 = 0$ totalizes without solving (GIN-PROP-016). As a machine operation, subtraction is $a + \neg b + 1$ in a ripple adder; the final carry is the *no-borrow* bit, i.e. the comparison $a \geq b$ (verified exhaustively, GIN-IMP-003).

4. Multiplication: $2 \times 3 = 6$

$2 + 2 + 2$ and $3 + 3$ are different derivations with the same value; commutativity is a theorem (GIN-PROP-013). Multiplication is: repeated addition (primitive recursion), the cardinality of a Cartesian product, scaling, composition of scalings ($x \mapsto 2x$ then $x \mapsto 3x$), shift-and-add in binary (partial products of an array multiplier: 64 gates for 4×4 bits, GIN-EXP-007), and a non-regular relation on numerals (GIN-THM-005). **Why $2 \times 0 = 0 \times 2 = 0 \times 0 = 0$:** four derivations in GIN-PROP-012 — ring axioms (needs cancellation), Peano recursion (base case plus induction), cardinality ($\emptyset \times A = \emptyset$), empty sum.

5. Division: $6 / 3 = 2$, $0 / 1 = 0$, $1 / 0$, $0 / 0$

Division is the converse of multiplication: $a / b = c \Leftrightarrow b \cdot c = a$. Six readings of $6/3 = 2$: inverse multiplication ($3 \cdot 2 = 6$); quotient and remainder ($6 = 2 \cdot 3 + 0$); partition (6 objects into 3 equal groups of 2); repeated subtraction ($6 - 3 - 3 = 0$: two steps); ratio (6 :

$3 = 2 : 1$; field operation ($6 \cdot 3^{-1}$). Machine readings: binary long division ($110_2 \div 11_2 = 10_2$, a restoring divider row by row), x86 `idiv`, AArch64 `sdiv`, RISC-V `divw` (GIN-OBS-007), Knuth's Algorithm D on limbs.

The admissibility theorem (GIN-THM-001): $\text{Sol}(a, b)$ is $\emptyset$ or $c_0 + \text{Ann}(b)$. Hence

expression	equation	image condition $a \in bR$	kernel condition $\text{Ann}(b) = 0$	outcome
$6 / 3$	$3c = 6$	$\checkmark$	$\checkmark$	2
$0 / 1$	$1c = 0$	$\checkmark (0 \in R)$	$\checkmark (1 \text{ is a unit})$	0
$1 / 0$	$0c = 1$	$\times (0 \cdot R = \{0\})$	$\times$	no solution (image failure)
$0 / 0$	$0c = 0$	$\checkmark$	$\times (\text{Ann}(0) = R)$	non-unique (kernel failure)
$x / 0, x \neq 0$	$0c = x$	$\times$	$\times$	no solution
$0 / x, x \neq 0$	$xc = 0$	$\checkmark$	$\checkmark$	0
$x / x, x \neq 0$	$xc = x$	$\checkmark$	$\checkmark$	1
$7 / 2 \text{ in } \mathbb{Z}$	$2c = 7$	$\times (7 \notin 2\mathbb{Z})$	$\checkmark$	no solution
$2 / 4 \text{ in } \mathbb{Z}/6$	$4c = 2$	$\checkmark (\gcd 2 \mid 2)$	$\times (\text{Ann}(4) = \{0, 3\})$	non-unique: $\{2, 5\}$

Symbolic vs numeric. As rational functions $x/x = 1$; as an expression evaluated at $x = 0$ it is $0/0$. Simplification must keep $x \neq 0$ (GIN-PROP-014). **Limits** answer a different question: $\lim_{x \rightarrow 0} x/x = 1$ is about a function near 0, not about the number $0/0$.

Machine floating point. binary64: $1/0 = +\infty$ (`divideByZero`), $1/-0 = -\infty$, $0/0 = \text{NaN}$ (`invalid`). These are *special data*, not solutions: $0 \times \infty = \text{NaN}$ (GIN-PROP-015). **Exact rationals** (Python `fractions`, `ginsdk` domain Q) never round, and raise on zero divisors.

Algebraic x: in $Q(\sqrt{2})$, division by nonzero elements is admissible (it is a field).

6. Extensions: what grammar changed?

RESULTS.md GIN-THM-004 (the trilemma table) and GIN-THM-006 (pairs): every system that gives $1/0$ a value gives up ring laws (projective line, wheels, extended reals), the converse reading (IEEE, meadows, `mathlib`, AArch64, RISC-V), or $0 \neq 1$ (the zero ring); in the pair arithmetic all indeterminate forms ($0/0, 0 \cdot \infty, \infty \pm \infty, \infty/\infty$) evaluate to the single null pair that cannot be an element.

7. Later operations

Operation	Converse of	Admissibility	Notes
<code>mod, //</code>	Euclidean division $a = qb + r, 0 \leq r < b $	$b \neq 0$	a <i>selection</i> : among the solutions of $a = qb + r$, the condition $0 \leq r < b $ picks one; languages pick differently (floor in Python, truncation in C/Java/x86)
$^$ (integer exponent)	$-$ (an action of $\mathbb{N}$ or $\mathbb{Z}$ on a monoid)	negative exponent needs an inverse	exponents are not ring elements (GIN-NEG-013)
$\sqrt{}$	squaring	$\mathbb{R}$: $a \geq 0$; two solutions $\rightarrow$ principal branch	$\mathbb{Q}$: $\sqrt{2}$ has no solution (irrational)
<code>log</code>	<code>exp</code>	$\mathbb{R}$: $a > 0$; $\mathbb{C}$: $a \neq 0$, infinitely many $\rightarrow$ branch	e^c is never 0, in $\mathbb{R}$ or $\mathbb{C}$
<code>gcd</code>	$-$	total on $\mathbb{Z}^2$ ($\gcd(0, 0) = 0$ by convention)	Euclid system, GIN-PROP-040
<code>lcm</code>	$-$	total	$\text{lcm}(a, b) = ab /\gcd(a, b)$ for $ab \neq 0$
<code>factorial</code>	$-$	$n \in \mathbb{N}$	Γ extends it, with poles at non-positive integers

Results Ledger

Every entry carries one category from the ladder of claims (RESEARCH_CHARTER.md §3), a status, and a relationship to prior work.

“Proved here” means the proof below was written by the session-1 collaborator and the exact statement is checked exhaustively on small cases by `experiments/check_theorems.py` (results in `experiments/results/checks.json`). No proof has been externally reviewed. **Nearly everything here is classical or elementary.** The default assumption for any arithmetic fact is that it is known; novelty is discussed in §G.

Notation: `DEFINITIONS.md`. b = base, D = digit set, val = value of a numeral, addr = canonical numeral, Sol = solution set of a converse problem, $\text{Ann}(b) = \{t : bt = 0\}$.

A. Numerals and the successor grammar

GIN-PROP-001 Numeral $\neq$ number: the value map and its non-injectivity (*Proposition; classical, proof written here*)

For standard base $b \geq 2$ the value map $\text{val}: \{0, \dots, b-1\}^* \rightarrow \mathbb{N}$ is surjective, and $\text{val}(u) = \text{val}(v)$ iff u and v differ only by leading zeros. Consequently exactly $L - |\text{addr}(n)| + 1$ words of length $\leq L$ denote n (for $|\text{addr}(n)| \leq L$).

Proof. Leading zeros do not change the value because $\llbracket 0 \rrbracket(0) = b \cdot 0 + 0 = 0$, so $\llbracket 0w \rrbracket(0) = \llbracket w \rrbracket(0)$. Conversely let u, v be words without leading zeros and $\text{val}(u) = \text{val}(v) = n$. If $n = 0$ both are empty (a non-empty word without leading zero has value $\geq b^{\{|w|-1\}} \geq 1$). Otherwise $b^{\{|u|-1\}} \leq n < b^{\{|u|\}}$ and likewise for v , so $|u| = |v|$; the last digits are both the residue $n \bmod b$, and removing them leaves words of equal value $(n - d)/b$; induct on length. Surjectivity: the residue algorithm $d = n \bmod b$, $n \leftarrow (n - d)/b$ strictly decreases $n > 0$ and stops at 0. ■

Check. All 6,303 pairs of words up to length 5 ($b = 2$) and 3 ($b = 3, 4$). *Experiment.* GIN-EXP-001.

GIN-PROP-002 Zero’s canonical numeral is the empty word (*Proposition; elementary; a REINTERPRETATION of CGT-DEF-006*)

In every digit grammar, $\text{addr}(0) = \varepsilon$. In standard base b the words denoting 0 are exactly $\varepsilon, 0, 00, 000, \dots$; the numeral “0” is the shortest *non-empty* one, i.e. a conventional non-canonical spelling.

Proof. ε is the shortlex-least word and $\llbracket \varepsilon \rrbracket(0) = 0$. The second claim is GIN-PROP-001 with $n = 0$. ■

Meaning. The zero digit plays two roles that the grammar separates: a **placeholder** inside numerals ($101 \neq 11$) and a **spelling** of the empty address. Bijective notation (GIN-PROP-004) needs neither: it has no zero digit and writes 0 as ε .

GIN-PROP-003 The cost of Peano arithmetic (*Proposition; elementary, proved here*)

Under the primitive-recursive definitions of GIN-DEF-010, $\text{add}(m, n)$ uses exactly $n + 1$ rule applications and $\text{mul}(m, n)$ uses exactly $mn + 2n + 1$. Hence on inputs whose binary numerals have k bits, unary addition costs $\Theta(2^k)$ rule applications against $\Theta(k)$ bit operations for binary addition: exponential in the length of the numeral.

Proof. add : one rule per S peeled from n plus the base rule. mul : by induction; $\text{mul}(m, 0)$ costs 1; $\text{mul}(m, S n)$ costs $1 + (mn + 2n + 1) + (m + 1) = m(n+1) + 2(n+1) + 1$, using the add cost with second argument m . ■

Check. $m, n < 12$. *Experiment.* GIN-EXP-001 table ($n = 2^{19} - 1$: 524,288 rules vs 20 bit operations).

GIN-PROP-004 Residue digit systems; bijective numeration (*Proposition; classical: Smullyan 1961 “k-adic”, Forslund 1995; proof written here*)

Let $b \geq 2$ and let D be a complete residue system mod b (or $b = 1$, $D = \{1\}$).

1. If a word w denotes n , its last digit is the unique $d \in D$ with $d \equiv n \pmod{b}$ and its prefix denotes $(n - d)/b$. So the words denoting n are determined by the residue algorithm up to prefixes that denote 0.
2. **Standard** ($D = \{0, \dots, b-1\}$) and **balanced ternary** ($b = 3$, $D = \{-1, 0, 1\}$): every $n \in \mathbb{N}$ (resp. $\mathbb{Z}$) has a canonical numeral and the words denoting n are exactly $0^k \cdot \text{addr}(n)$.
3. **Bijective** ($D = \{1, \dots, b\}$): val is a bijection $D^* \rightarrow \mathbb{N}$.

Proof. (1) $\text{val}(w'd) = b \cdot \text{val}(w') + d$. (2) The residue algorithm terminates: for standard digits as in GIN-PROP-001; for balanced ternary $|(n - d)/3| \leq (|n| + 1)/3 < |n|$ for $|n| \geq 1$. A prefix denoting 0 must end in the digit $\equiv 0$, which is 0, and recursively consists of zeros. (3) Every non-empty word of positive digits has positive value, so only ε denotes 0; for $n \geq 1$ the algorithm gives $n \mapsto (n - d)/b < n$ with $d \in \{1, \dots, b\}$, so it terminates, and by (1) the word is unique. ■

Check. Bijective bases 1, 2, 3, 5 exhaustively by length (all values distinct and forming an initial segment of $\mathbb{N}$). *Experiment.* GIN-EXP-001.

Counterexample to dropping the hypothesis. $b = 2$, $D = \{-1, 2\}$ is a complete residue system but the non-empty word $(-1)(2)$ denotes 0, so uniqueness fails. Ordinary binary does not terminate on negative numbers: GIN-PROP-008.

GIN-PROP-005 Compilation and the concatenation law (*Proposition; classical, restated as a CGT-DEF-007 transition monoid*)

For a digit grammar (b, D) and words u, v : $\llbracket w \rrbracket = (x \mapsto b^{|w|}x + \text{val}(w))$, hence $\llbracket uv \rrbracket = \llbracket v \rrbracket \circ \llbracket u \rrbracket$ and $\text{val}(uv) = \text{val}(u) \cdot b^{|v|} + \text{val}(v)$. The transition monoid is $\{x \mapsto b^k x + c\}$.

Proof. Induction on $|w|$ using $(x \mapsto b x + d) \circ (x \mapsto b^k x + c) = x \mapsto b^{k+1} x + (bc + d)$. ■

Remark (CGT-DEF-011). The compiled element $(b^{|w|}, \text{val}(w))$ has $\Theta(|w|)$ bits, so digit grammars are **not** arithmetically compilable in CGT's $O(1)$ -word sense for unbounded words. Contrast GIN-PROP-020.

GIN-PROP-006 CGT's heap numeration is bijective binary, shifted by one (*Proposition; elementary, proved here; likely folklore, novelty unverified*)

Let $\varphi(x) = x + 1$. The bijective base-2 grammar (digits 1, 2; moves $x \mapsto 2x+1$, $x \mapsto 2x+2$; root 0) is conjugate by φ to the CGT heap grammar on binary trees (moves L: $y \mapsto 2y$, R: $y \mapsto 2y + 1$; root 1): $\varphi \circ \llbracket 1 \rrbracket = L \circ \varphi$ and $\varphi \circ \llbracket 2 \rrbracket = R \circ \varphi$. Consequently the bijective base-2 numeral of n , with $1 \mapsto L$ and $2 \mapsto R$, is the CGT heap address of node $n + 1$, i.e. the binary numeral of $n + 1$ with its leading 1 removed.

Proof. $(2x + 1) + 1 = 2(x + 1)$ and $(2x + 2) + 1 = 2(x + 1) + 1$. By induction on words, $\varphi \circ \llbracket w \rrbracket = \llbracket h(w) \rrbracket \circ \varphi$ with $h(1) = L$, $h(2) = R$; evaluate at 0. Both addresses are unique words (GIN-PROP-004(3); the heap address of $m \geq 1$ is unique), so they correspond. ■

Check. $n < 300,000$. *Meaning.* The address insight that motivated CGT (an address carries operations; CGT-DEF-006, CGT-THM-003, CGT-THM-006) is, for numbers, positional notation itself.

GIN-PROP-007 Positional numerals are straight-line programs for unary terms (*Proposition; classical: addition chains, Knuth TAOCP vol. 2 §4.6.3; SLPs, Charikar et al. 2005*)

For $n \geq 1$ with binary numeral of length k and popcount ν , the unary word S^n (equivalently a^n) is generated by a straight-line program with $k + \nu - 1 \leq 2k - 1$ productions: $A_1 \rightarrow a$, and for each later bit a doubling production $A \rightarrow A'A'$ followed, for bit 1, by $A \rightarrow A'a$. This program has exactly the shape of the exponent program for x^n (GIN-PROP-041): $k - 1$ squarings and $\nu - 1$ multiplications. Conversely an SLP of size m for a^n yields an addition chain for n of length at most m (literature).

Proof. Horner's rule for the binary numeral: reading bit β maps the current length ℓ to $2\ell + \beta$; doubling is concatenation of the word with itself, the bit 1 appends one letter. Count the productions. ■

Meaning. Positional notation is CGT's M-SHARE mechanism applied to the successor grammar: exponential compression of unary numerals (GIN-REI-008).

GIN-PROP-008 Two's complement is the truncated 2-adic expansion (*Proposition; classical; proof written here*)

For every integer n and width w , the first w digits $d_0, \dots, d_{w-1}$ of the base-2 residue expansion of n ($d = x \bmod 2$, $x \leftarrow (x - d)/2$) are the bits of $n \bmod 2^w$ — the w -bit two's-complement encoding when $-2^{w-1} \leq n < 2^{w-1}$. For $n < 0$ the expansion never terminates: the state reaches -1 , a fixed point of $x \mapsto (x - 1)/2$, after which every digit is 1. The infinite word $\dots 111d_{w-1} \dots d_0$ is the 2-adic expansion of n .

Proof. After w steps $n = \sum_{i < w} d_i 2^i + 2^w x_w$, so $\sum d_i 2^i \equiv n \pmod{2^w}$ and lies in $[0, 2^w)$. For $x < 0$, $x \mapsto \lfloor x/2 \rfloor$ is non-decreasing toward -1 and fixes -1 , where the digit is 1. ■

Check. All representable n for widths 1–12. *Meaning.* The grammar of standard binary *cannot* reach negative numbers in finitely many moves; the hardware's answer, fixed width, is a quotient (GIN-DEF-017, repair 4) that keeps only the first w moves.

GIN-PROP-009 Long division is a finite transition system (*Proposition; classical*)

Expanding p/q ($0 \leq p < q$, lowest terms) in base b runs the transition system on remainders $r \mapsto (b \cdot r) \bmod q$, emitting digit $\lfloor b \cdot r / q \rfloor$. The expansion terminates iff every prime factor of q divides b ; otherwise it is eventually periodic, with period at most $q - 1$.

Proof. The state space $\{0, \dots, q-1\}$ is finite, so the trajectory is eventually periodic; it reaches 0 iff $p/q = m/b^k$ for some k iff $q \mid b^k$ (lowest terms) iff the primes of q divide b . ■

Check. Bases 2, 3, 10, 12, denominators < 200 . *Example.* $1/3 = 0.(3)$ in base 10 and $0.(01)$ in base 2; $1/10 = 0.0(0011)$ in base 2 — which is why 0.1 is not a binary64 number (GIN-PROP-031).

GIN-PROP-010 The cost of the successor in binary (*Proposition; the binary counter is textbook (CLRS §17.1); closed form proved here*)

Counting from 0 to N in binary flips exactly $2N - \text{popcount}(N)$ bits. The successor therefore costs 2 bit flips amortized and $\lfloor \log_2 N \rfloor + 1$ in the worst case (at $N = 2^k - 1 \rightarrow 2^k$).

Proof. The increment $x \rightarrow x + 1$ turns the $t(x)$ trailing ones into zeros and one zero into a one: it flips $t(x) + 1$ bits and changes the popcount by $1 - t(x)$. Summing over $x = 0, \dots, N-1$: $\text{flips} = \Sigma(t + 1) = \Sigma(2 - (1 - t)) = 2N - (\text{popcount}(N) - \text{popcount}(0))$. ■

Check. $N \leq 65,536$; GIN-EXP-002 to 131,072 (amortized 2.0000, worst 18).

B. The segment notation $0\{1,2,3\}4$

GIN-REI-010 Verdict on the segment notation (*Reinterpretation; notation proposed by A. Kadaan, formalized in GIN-DEF-012*)

Formalized, $a\{I\}b$ denotes the segment $[a, b]$ of the successor transition system with boundary $\{a, b\}$ outside the braces and interior I inside. The notation has **formal value** in three precise senses: (i) it is *checkable* — it carries redundant information (I is determined by a and b), so ill-formed instances such as $0\{1,3\}4$ are detectable, with a diagnosis (`ginsdk.peano.parse_segment`); (ii) it *displays* the transition-system distinction between boundary points (one admissible move: S at a , P at b) and interior points (both moves admissible); (iii) it generalizes to any transition system as “endpoints plus the points strictly between them along a path”. It has **no new mathematical content**: as a set it is the closed interval $[a, b]$, and as a structure it is the path graph on $b - a + 1$ vertices with marked endpoints. It is notation, not theory.

C. Converses: division, zero and admissibility

GIN-THM-001 The structure of a converse in a commutative ring (*Theorem; standard algebra, proof written here*)

Let R be a commutative ring and $a, b \in R$. The solution set $\text{Sol}(a, b) = \{c \in R : bc = a\}$ is either empty or a coset $c_0 + \text{Ann}(b)$. Hence:

1. **image condition**: $\text{Sol}(a, b) \neq \emptyset$ iff $a \in bR$;
2. **kernel condition**: when non-empty, $|\text{Sol}(a, b)| = |\text{Ann}(b)|$; the quotient is unique iff $\text{Ann}(b) = \{0\}$.

Proof. If $c_0 \in \text{Sol}$ then $bc = a \Leftrightarrow b(c - c_0) = 0 \Leftrightarrow c - c_0 \in \text{Ann}(b)$. ■

Check. 30 finite commutative rings ($\mathbb{Z}/n$ for $n \leq 24$, five products $\mathbb{Z}/m \times \mathbb{Z}/k$, and $\mathbb{F}_2[x]/(x^3)$), all 5,737 pairs (a, b) . The same statement holds for any homomorphism of abelian groups (solution sets of linear equations are empty or cosets of the kernel).

GIN-COR-001 Why $1/0$ and $0/0$ fail differently, and why $0/1 = 0$ (*Corollary; standard*)

In a field F (and in $\mathbb{Z}, \mathbb{Q}, \mathbb{R}, \mathbb{C}$):

- $b \neq 0$: $\text{Ann}(b) = \{0\}$ and $bF = F$, so a/b is admissible for every a . In particular **$0/1 = 0$** : the image condition holds because $0 \in 1 \cdot F$, and the kernel condition holds because 1 is a unit; the unique solution of $1 \cdot c = 0$ is $c = 0$.
- **$1/0$** : $0 \cdot F = \{0\}$ (GIN-PROP-012), so the image condition fails: $0 \cdot c = 1$ has **no solution** (it would require $0 = 1$). An *image failure*.
- **$0/0$** : the image condition holds ($0 \in 0 \cdot F$), but $\text{Ann}(0) = F$, so **every** c solves $0 \cdot c = 0$. A *kernel failure*: not “no answer” but “no single answer”.
- **$x/0$ for symbolic x** : no solution where $x \neq 0$, every c where $x = 0$ — a case split that symbolic simplification must not erase (GIN-PROP-014).

Remark. In the zero ring $\{0\}$ (where $1 = 0$) the quotient $0/0$ is admissible and equals 0 ; this is the only ring in which division by zero is admissible (GIN-THM-003).

GIN-THM-002 Division in $\mathbb{Z}/n$ (*Theorem; standard number theory, proof written here*)

In $\mathbb{Z}/n$ with $g = \gcd(b, n)$: $\text{Sol}(a, b) \neq \emptyset$ iff $g \mid a$, and then $|\text{Sol}(a, b)| = g$, the solutions forming $c_0 + (n/g)\mathbb{Z}/n$. Exactly $n \cdot \varphi(n)$ of the n^2 pairs (a, b) have a unique quotient, and $\Sigma_{b=0}^{n-1} n/\gcd(b, n)$ are solvable.

Proof. By Bézout, $b\mathbb{Z}/n = g\mathbb{Z}/n$, which gives the image condition. $bt \equiv 0 \pmod{n}$ iff $(n/g) \mid (b/g)t$ iff $(n/g) \mid t$, since $\gcd(b/g, n/g) = 1$; so $\text{Ann}(b) = (n/g)\mathbb{Z}/n$ has g elements. A unique quotient requires $g = 1$ (b a unit), and then all n values of a work: $n \cdot \varphi(n)$ pairs. For each b , the solvable a are the n/g multiples of g . ■

Check. $n \leq 60$ exhaustively; GIN-EXP-003 to $n = 64$ (0 violations; unique share $= \varphi(n)/n$).

GIN-THM-003 No non-trivial ring can invert zero (*Theorem; standard*)

If R is a ring with 1 in which $0 \cdot z = 1$ for some z , then $R = \{0\}$.

Proof. $0 \cdot z = 0$ (GIN-PROP-012(i)), so $1 = 0$, and every $x = x \cdot 1 = x \cdot 0 = 0$. ■

Check. $\mathbb{Z}/n$, $n < 60$: an inverse of 0 exists only for $n = 1$.

GIN-THM-004 The extension trilemma for $1/0$ (*Reinterpretation: a synthesis of cited facts; the trilemma formulation is GiN's*)

By GIN-THM-003, any system that gives $1/0$ a value must give up at least one of: **(A)** the ring laws that force $0 \cdot c = 0$ (distributivity with additive inverses), **(B)** the *converse reading* of division (that a/b denotes a c with $b \cdot c = a$), or **(C)** $0 \neq 1$. Every system in the literature and in hardware takes route A, B, or *signals* instead of answering:

System	1/0	0/0	Route	What gives way
$\mathbb{Q}, \mathbb{R}, \mathbb{C}$ (fields)	no solution	non-unique	restriction	division is defined only for $b \neq 0$
projective line $\mathbb{Q} \cup \{\infty\}$, Riemann sphere $\mathbb{C} \cup \{\infty\}$	∞	undefined	A	$0 \cdot \infty, \infty \pm \infty, \infty/\infty, 0/0$ undefined; no field structure; no order (GIN-THM-006)
extended reals $\mathbb{R} \cup \{\pm\infty\}$	usually undefined ($a/\pm\infty = 0$ is defined)	undefined	A	$0 \cdot (\pm\infty)$ and $\infty - \infty$ undefined; $\pm\infty$ are not field elements
IEEE 754 floating point	$\pm\infty$, flag divideByZero (sign from the operands, including the sign of zero)	NaN, flag invalid	B	$0 \times \infty = \text{NaN}$, so ∞ is not a solution of $0 \times c = 1$ (GIN-PROP-015); $\text{NaN} \neq \text{NaN}$
meadows (Bergstra & Tucker 2007)	0	0	B	$x \cdot x^{-1} = 1$ only for $x \neq 0$; ring axioms kept
Lean's mathlib ($0^{-1} = 0$)	0	0	B	theorems such as $a \cdot a^{-1} = 1$ carry the hypothesis $a \neq 0$
wheels (Carlström 2004)	$/0$, a distinguished element	$\perp$ (absorbing)	A	$0 \cdot x = 0$ fails in general; distributivity weakened to $(x+y)z + 0z = xz + yz$
AArch64 SDIV/UDIV	0, no signal	0	B (totalization)	the result is not a quotient of anything
RISC-V DIV/DIVU	$-1 / 2^w - 1$, remainder = dividend	same	B (totalization)	as above; matches the natural output of a restoring divider (GIN-PROP-023)
x86-64 IDIV, Java, Rust, Go, Python, JS BigInt	trap / exception / panic	same	signal	no value is produced
ISO C	undefined behaviour	same	none	the program has no meaning (GIN-OBS-008)
limits	$\lim_{x \rightarrow 0 \pm} 1/x = \pm\infty$ (not a real limit)	$\lim f/g$ depends on f and g	—	a question about <i>functions</i> , not numbers

Facts about meadows, wheels and mathlib are taken from the cited sources and secondary literature (LITERATURE_REVIEW.md §7); hardware and language behaviour is measured (GIN-EXP-004) or cited to specifications (MACHINE_MODEL.md).

GIN-THM-006 Fractions as pairs: ∞ can be adjoined, $0/0$ cannot (*Theorem; elementary, proved here; the projective line is classical, and keeping $(0,0)$ as an absorbing element is Carlström's wheel of fractions; novelty of the pair-level statement unverified*)

On $\mathbb{Z}^2$ let $(a, b) \sim (c, d)$ iff $ad = bc$, and define $(a,b) + (c,d) = (ad + bc, bd)$, $(a,b) \cdot (c,d) = (ac, bd)$, $(a,b)/(c,d) = (ad, bc)$.

- $\sim$ is reflexive and symmetric on $\mathbb{Z}^2$. On $\mathbb{Z}^2 \setminus \{(0,0)\}$ it is an equivalence relation whose classes are the lines through the origin: the projective line $\mathbb{Q} \cup \{\infty\}$, where ∞ is the single class of all $(a, 0)$, $a \neq 0$. On $\mathbb{Z} \times (\mathbb{Z} \setminus \{0\})$ the classes are $\mathbb{Q}$.
- $(0, 0)$ is related to **every** pair, so on $\mathbb{Z}^2$ transitivity fails: $(1,0) \sim (0,0) \sim (0,1)$ but $(1,0) \not\sim (0,1)$. Admitting “ $0/0$ ” as a pair would identify everything.
- The operations are well defined on classes (scaling an operand by $k \neq 0$ scales the result by k), with result the null pair $(0,0)$ exactly in the cases $\infty + \infty$ (hence $\infty - \infty$, since $-\infty = \infty$ here), $0 \cdot \infty$, $0/0$ and ∞/∞ ; every other combination of elements of $\mathbb{Q} \cup \{\infty\}$ is defined. In particular **$1/0 = \infty$** , $x/0 = \infty$ for $x \neq 0$, $1/\infty = 0$, $0/\infty = 0$.

So in the arithmetic of pairs, **every indeterminate form computes the same object — the null pair — and the null pair is exactly the one pair that cannot be an element.**

Proof. (1) Reflexivity and symmetry are immediate. Transitivity for $p \sim q \sim r$ with $q \neq (0,0)$: if $q = (c, d)$ with $d \neq 0$, then $ad = bc$ and $cf = de$ give $adf = bcf = bde$, so $d(af - be) = 0$ and $af = be$. If $d = 0$ then $c \neq 0$; $ad = bc$ gives $b = 0$, and $cf = de$ gives $f = 0$, so $af = 0 = be$. A pair is $\sim$ to $(1, 0)$ iff its second entry is 0. (2) $a \cdot 0 = 0 \cdot b$. (3) Each operation multiplies componentwise by k when one operand is scaled by k , so classes go to classes and the null pair to itself. Null sum: $bd = 0$ and $ad + bc = 0$; if $b = 0$ then $a \neq 0$ forces $d = 0$, i.e. both are ∞ (and then the first entry is 0 automatically); symmetrically if $d = 0$. Null product: $ac = 0$ and $bd = 0$ with neither operand null: one factor has first entry 0 (it is 0) and the other second entry 0 (it is ∞). Null quotient: $ad = 0$ and $bc = 0$: either $a = c = 0$ ($0/0$) or $b = d = 0$ (∞/∞) — the mixed cases would make an operand null. ■

Check. All pairs in $[-4, 4]^2$, 19,200 operations, including well-definedness under scaling.

GIN-PROP-011 $0/1 = 0$ as a complete case study (*Proposition; trivial; included for the narrative*)

See GIN-COR-001. The contrast with $1/0$ is not a swap of the symbols 0 and 1 but a change of *which side* zero is on: a zero numerator tests the image condition with the always-present element $0 \in bR$; a zero denominator shrinks the image to $\{0\}$ and the kernel to everything.

GIN-PROP-012 Why $0 \cdot a = 0$, derived four ways (*Proposition; classical; multi-framework presentation is GiN's*)

- Ring axioms.** $0a = (0 + 0)a = 0a + 0a$; adding $-(0a)$ gives $0a = 0$. Uses distributivity *and* additive inverses (cancellation). In a semiring without cancellation, $0 \cdot a = 0$ does not follow and is taken as an axiom (absorption).
- Peano recursion.** $a \cdot 0 = 0$ is the base equation of mul. $0 \cdot a = 0$ by induction on a : $0 \cdot 0 = 0$, $0 \cdot S(a) = 0 \cdot a + 0 = 0 + 0 = 0$.
- Cardinality.** $|\emptyset \times A| = |\emptyset| = 0$ because there are no pairs with a first component from $\emptyset$.
- Repeated addition.** $a \cdot 0$ is the sum of zero copies of a : the empty sum, which is the additive identity 0.

GIN-PROP-013 Why $2 \times 3 = 3 \times 2$ (*Proposition; classical*)

$2 + 2 + 2$ and $3 + 3$ are different words (different derivations) with the same value. Two proofs that they must agree: (i) induction on the Peano definitions (lemmas $S(m) \cdot n = m \cdot n + n$ and $m \cdot n = n \cdot m$); (ii) the bijection $(x, y) \mapsto (y, x)$ between $A \times B$ and $B \times A$, so $|A| \cdot |B| = |B| \cdot |A|$. The equality is a theorem about the semantics, not a fact visible in the syntax.

GIN-PROP-014 Symbolic simplification is not evaluation (*Proposition; elementary*)

In the field $\mathbb{Q}(x)$ of rational functions, $x/x = 1$ is an identity. As an expression evaluated at points of $\mathbb{Q}$, x/x is admissible exactly on $\mathbb{Q} \setminus \{0\}$, and at 0 it is the kernel failure $0/0$. Cancelling a common factor g preserves values on $\{g \neq 0\}$ and silently enlarges the domain; a correct simplifier must record $g \neq 0$ as a side condition. Likewise $(x^2 - 1)/(x - 1) = x + 1$ only on $x \neq 1$.

Implementation. `ginsdk.expr` (domain `symbolic`) keeps every divisor's condition before cancellation and reports the failure at each excluded point (GIN-EXP-003).

GIN-PROP-015 IEEE division by zero is not the converse of IEEE multiplication (*Proposition; elementary consequence of IEEE 754*)

For finite nonzero x , IEEE 754 gives $x/(\pm 0) = \pm\infty$ with `divideByZero`, and $0/0 = \text{NaN}$ with `invalid`. But $0 \times (\pm\infty) = \text{NaN}$ (`invalid`), and $0 \times c = \pm 0$ for finite c , so no IEEE datum c satisfies $0 \otimes c = 1$. IEEE's $\pm\infty$ is a limit-motivated value whose sign records the side from which the zero was approached (the sign of zero; Kahan 1987), not a solution of the converse problem.

Check. Four formats, both signs; hardware agreement GIN-IMP-001.

GIN-PROP-016 Truncated subtraction is a totalization, not a converse (*Proposition; trivial*)

$m \dot{-} n = \max(m - n, 0)$ is total on $\mathbb{N}$ and agrees with subtraction where subtraction is admissible, but $0 \dot{-} 1 = 0$ is not a solution of $1 + c = 0$. Monus is repair 5 (totalization), exactly like $x / \emptyset = \emptyset$ in meadows.

GIN-PROP-017 The admissibility closures of $\mathbb{N}$ (*Proposition; classical constructions restated*)

$\mathbb{Z}$ is the Grothendieck group of $(\mathbb{N}, +)$: pairs (a, b) modulo $(a, b) \sim (c, d)$ iff $a + d = b + c$, so the element “ $a - b$ ” is the unanswered converse problem kept as data. $\mathbb{Q}$ is the same construction for multiplication on $\mathbb{Z} \times (\mathbb{Z} \setminus \{0\})$ (GIN-THM-006). In each case the new carrier is the admissibility closure (GIN-DEF-019) for the converse in question, and the exclusion of the zero denominator is forced (GIN-THM-006(2)).

GIN-HIST-001 Brahmagupta's rules for zero (628 CE) (*Historical result; via secondary sources*)

In the *Brāhmasphuṭasiddhānta* Brahmagupta defined zero as the result of subtracting a number from itself and gave arithmetic rules for zero, including $0/0 = 0$; for a nonzero number divided by zero he described a fraction with zero denominator. Bhāskara II later called a quantity with zero divisor *khahara*. (MacTutor; Plofker 2009.)

GiN's reading (interpretation). By GIN-COR-001, $0/0 = 0$ is not *derivable*: every c solves $0 \cdot c = 0$. But $0/0 = 0$ is a consistent *selection* — the same totalization that meadows and Lean's `mathlib` adopt (GIN-THM-004). The modern objection is that it cannot be the converse answer, not that it is inconsistent.

D. Representation, circuits and machines

GIN-PROP-020 The carry monoid and carry-lookahead (*Proposition; known in substance: Ladner & Fischer 1980, Brent & Kung 1982; CGT reading is GiN's*)

The carry automaton of binary addition has transition monoid $\{K, P, G\}$ (GIN-DEF-041), with composition “low position, then high position”: the result is the high element if it is K or G , and the low element if the high element is P . P is the identity, and every element is idempotent. The carry into position $i + 1$ is the image of the input carry under the composite of positions $0 \dots i$. Since the composition is associative, all carries are prefix products and can be computed by any parallel-prefix network, in depth $O(\log n)$.

Proof. Direct from the three maps $K = \text{const } 0$, $P = \text{id}$, $G = \text{const } 1$ on $\{0, 1\}$. ■ *Check.* Associativity and identity exhaustively; prefix products vs sequential carries on 500 random additions (tests); prefix adders exhaustively for $n \leq 8$ (GIN-EXP-006).

CGT reading (GIN-REI-005). Any word of digit pairs compiles to one of three elements: the carry automaton is **arithmetically compilable** in the sense of CGT-DEF-011, and carry-lookahead is that compilation computed in parallel.

GIN-PROP-021 Ripple-carry cost (*Proposition; elementary, for the construction in `ginsdk.circuits`*)

The n -bit ripple-carry adder (half adder at position 0, full adders XOR/XOR/AND/AND/OR above) has exactly $5n - 3$ gates and depth $2n - 1$.

Proof. $2 + 5(n - 1)$ gates. The carry out of position 0 has depth 1 and each full adder adds 2 (AND then OR) to the carry path, so the carry out of position $n - 1$ has depth $2n - 1$; sum bits have depth $\leq 2n - 2$. ■ *Check.* $n < 200$.

GIN-PROP-023 A divider that never tests the divisor returns all ones (*Proposition; proved here; consistent with the RISC-V specification's stated rationale*)

The `n`-bit restoring array divider of `ginsdk.circuits` (no test of the divisor anywhere in the circuit) outputs quotient $2^n - 1$ and remainder equal to the dividend when the divisor is 0. These are exactly the DIVU/REMU results the RISC-V M extension specifies.

Proof. Row i computes $T = 2R + a_i$ and keeps $T - D$ if $T \geq D$. With $D = 0$, $T \geq D$ always, so every quotient bit is 1, and $R \leftarrow T$, so after the row for bit i , $R = \lfloor a/2^i \rfloor < 2^{n-i} \leq 2^n$ (the n -bit register suffices); after the last row $R = a$. ■

Check. $n \leq 6$, all dividends; GIN-EXP-007. *Prior work.* The RISC-V specification explains its choice by the simplicity of divider circuitry (“the natural result for simple unsigned divider implementations”); GiN supplies a proof and a gate-level check for one such implementation, not a new fact.

GIN-THM-005 Addition is finite-state; multiplication is not (*Theorem; known: Büchi 1960; Hodgson 1983; Khossainov & Nerode 1995; Blumensath & Grädel 2000; Gödel 1931; Church 1936*)

Fix a base $b \geq 2$ and read numerals least significant digit first, padded to equal length. (1) The graph of addition $\{(x, y, x + y)\}$ is recognized by a synchronous finite automaton (the carry automaton), so $(\mathbb{N}, +)$ is an automatic structure. (2) The graph of multiplication is not synchronous rational in any presentation in which addition is: otherwise $(\mathbb{N}, +, \times)$ would be automatic, every automatic structure has a decidable first-order theory, and the first-order theory of $(\mathbb{N}, +, \times)$ is undecidable.

Classical refinements. Multiplication by a fixed constant k is finite-state (the carry is bounded by k); remainder modulo a fixed m , read most significant digit first, is finite-state (states = residues, GIN-PROP-009); hence divisibility by m is a regular language in every base.

Meaning. The successor, addition, comparison, multiplication by constants and reduction modulo constants live in the class of **regular** operational grammars over numerals; general multiplication does not. The finite carry monoid of GIN-PROP-020 is the algebraic face of (1).

GIN-PROP-030 The least unrepresentable integer (*Proposition; classical*)

In a binary floating-point format with precision p and $\text{emax} \geq p$, every integer $0 \dots 2^p$ is representable and $2^p + 1$ is not.

Proof. An integer $m < 2^p$ is $m \cdot 2^0$ with an integer significand below 2^p ; $2^p = 2^{\{p-1\}} \cdot 2$. The numeral of $2^p + 1$ is $10\dots01$ with $p + 1$ significant bits. ■ *Check / experiment.* Found by search for all four formats (GIN-EXP-012): 17, 2049, 16,777,217, 9,007,199,254,740,993.

GIN-PROP-031 Which decimals are binary floating-point numbers (*Proposition; elementary, proved here*)

A decimal $d/10^k$ ($1 \leq d < 10^k$) is exactly representable in a binary format with $p \geq k$ and enough exponent range iff $5^k \mid d$. Exactly $2^k - 1$ of the $10^k - 1$ k -digit decimals are representable.

Proof. In lowest terms the denominator is $2^j 5^l$, and a binary fraction must have $j = 0$, i.e. $5^k \mid d$; then $d/10^k = m/2^k$ with $m < 2^k \leq 2^p$. The multiples of 5^k below $10^k = 2^k 5^k$ number $2^k - 1$. ■ *Check.* $k \leq 5$ in binary64 (1, 3, 7, 15, 31).

GIN-PROP-032 Floating-point addition is commutative but not associative (*Proposition; classical*)

Commutativity holds because each sum is the correctly rounded exact sum, which is commutative. Associativity fails: $(0.1 + 0.1) + 0.4 = 0.6000000000000001$ but $0.1 + (0.1 + 0.4) = 0.6$ in binary64. Of the 729 triples of one-digit tenths, 242 are non-associative (GIN-EXP-012).

GIN-IMP-001 The IEEE simulator agrees with this CPU, results and flags (*Implementation result*)

On x86-64 (SSE scalar arithmetic, gcc 13.3, fenv.h), `ginsdk.ieee` reproduces the hardware's results bit for bit (NaNs compared as NaNs) and all five exception flags on 40,300 binary32 and 40,300 binary64 cases (+, −, ×, ÷, √; random patterns, special values, subnormals, and a targeted tininess family), **after** modelling signaling NaNs (GIN-REJ-002). Scope: one host, one rounding attribute.

GIN-IMP-002 Machine models agree with measurement (*Implementation result*)

All 14 behaviours of `ginsdk.machine` that can be measured on the host (x86 traps for x/0 and INT_MIN/−1, truncation, Java, Rust debug and release, Go, Python, JS BigInt) agree with real executions (GIN-EXP-004). The AArch64 and RISC-V models are specification models only.

GIN-IMP-003 Circuits and limb arithmetic are correct on their test domains (*Implementation result*)

All adders exhaustively for $n \leq 8$ and randomly to $n = 64$; multiplier, subtractor and divider exhaustively for $n \leq 6$; limb arithmetic against Python `int` on thousands of random operands with limb sizes 8, 16, 32.

E. Number theory as transition systems

GIN-PROP-040 Euclid’s system and Lamé’s bound (*Historical result: Euclid, Elements VII.1-2; Lamé 1844*)

The Euclid system (GIN-DEF-050) preserves gcd, terminates because the second component strictly decreases, and the smallest pair $a > b > 0$ needing k division steps is (F_{k+2}, F_{k+1}) . Hence the number of steps is at most $\log_\varphi(a) + O(1)$.

Check. All pairs below 700 (step counts 1...13). *Experiment.* GIN-EXP-009: all pairs below 1,200.

GIN-PROP-041 The exponent’s numeral is the program (*Proposition; classical: Knuth TAOCP vol. 2 §4.6.3*)

Left-to-right binary exponentiation runs the exponent program (GIN-DEF-052): $\lfloor \log_2 e \rfloor$ squarings and $\text{popcount}(e) - 1$ multiplications; reading the move word back (M and $SM \mapsto 1$, $S \mapsto 0$) recovers the binary numeral of e . These counts coincide with the production counts of GIN-PROP-007.

Check. $e < 5,000$.

GIN-PROP-042 The Chinese remainder theorem as a converse problem (*Proposition; classical*)

The system $x \equiv r_i \pmod{m_i}$ asks for the converse of $x \mapsto (x \bmod m_i)$. The kernel modulo $L = \text{lcm}(m_i)$ is trivial, so the system is never non-unique mod L ; the image condition is pairwise compatibility $r_i \equiv r_j \pmod{\gcd(m_i, m_j)}$. The solution set is $\emptyset$ or a single residue class mod L .

Check. Two moduli below 16, all residues.

GIN-PROP-043 Primality tests as admissibility checks, and their counterexamples (*Proposition; classical: Fermat; Korselt 1899; Alford, Granville & Pomerance 1994; Miller 1976; Rabin 1980; Sorenson & Webster 2017*)

Fermat’s test $a^{n-1} \equiv 1 \pmod{n}$ is necessary for primality but not sufficient: $341 = 11 \cdot 31$ passes base 2; Carmichael numbers (Korselt: squarefree composite with $p - 1 \mid n - 1$ for all $p \mid n$) pass every coprime base, and there are infinitely many. The strong (Miller-Rabin) test with the first 13 prime bases is deterministic below $\psi_{13} = 3,317,044,064,679,887,385,961,981$.

Experiment. GIN-EXP-010 reproduces the published counts below 10^6 : 245 base-2 Fermat pseudoprimes, 46 strong base-2 pseudoprimes, 43 Carmichael numbers; Miller–Rabin agrees with the sieve on every $n \leq 10^6$.

F. Empirical observations

ID	Observation	Experiment
GIN-OBS-001	Peano addition of $n = 2^{19} - 1$ to itself takes 524,288 rule applications; binary addition 20 bit operations.	EXP-001
GIN-OBS-002	Mean longest carry chain for random n -bit additions is $\log_2 n - c$ with c between 0.66 and 0.84 for $n = 4 \dots 4096$; consistent with $\log_2 n + O(1)$ (Burks, Goldstine & von Neumann 1946; Knuth 1978). The constant depends on how a chain is counted.	EXP-002
GIN-OBS-003	The meaning table: $1/\emptyset$ has 7 distinct outcomes across 15 domains (no solution, ∞ , exception, trap, 0, -1 , undefined).	EXP-003
GIN-OBS-004	Constant $1/\emptyset$ is a compile-time error in Go and Rust, a warning then SIGFPE in C (gcc), and compiles silently then throws ArithmeticException in Java: the same sentence fails at different layers.	EXP-004
GIN-OBS-005	Go returns -2^{31} for <code>int32 -2^{31}-1</code> without a panic; Rust panics in both debug and release; x86 C traps; Java returns -2^{31} .	EXP-004
GIN-OBS-006	Python raises ZeroDivisionError for 1.0/0.0, where IEEE 754 (and C, Java, Go, JS) return $+\infty$.	EXP-004
GIN-OBS-007	The same <code>a / b</code> selects <code>idivl</code> (x86-64), <code>sdiv</code> (AArch64) and <code>divw</code> (RISC-V) for int, and <code>divsd</code> , <code>fdiv</code> , <code>fdiv.d</code> for double (clang 18 -O2).	EXP-004
GIN-OBS-008	In <code>int q = a / b; if (b == 0) return -1; return q;</code> clang 18 at -O2/-O3 deletes the zero test (the division already made <code>b = 0</code> undefined), while gcc 13.3 at -O2 keeps it (test + <code>cmove</code>).	EXP-004
GIN-OBS-009	This x86-64 host detects floating-point tininess after rounding : in a targeted family of 300 products just below the smallest normal number, the after-rounding model matches all hardware underflow flags and the before-rounding model mismatches 158 (binary32) / 159 (binary64).	EXP-005
GIN-OBS-010	Measured log-log slopes of 32-bit limb operations at 16,384 bits: add 1.0, schoolbook mul 2.0, Karatsuba 1.604 ($\rightarrow \log_2 3 \approx 1.585$), division 1.996; Euclid gcd ≈ 2 ; modular exponentiation ≈ 2.9 ($\rightarrow 3$).	EXP-008
GIN-OBS-011	Average Euclid steps for prime n equal $(12 \ln 2/\pi^2) \ln n + \text{Porter’s constant} - 1.000 \pm 0.009$ for $n = 1009 \dots 1,000,003$; the -1 is the counting convention (Porter’s τ_n includes the initial step $m \bmod n$ with $m < n$).	EXP-009
GIN-OBS-012	Pollard rho needs about $0.9\text{--}1.13\sqrt{p}$ steps to find a prime factor p (20 trials each, p of 10–26 bits).	EXP-010
GIN-OBS-013	Numeral evaluation: Horner and balanced-schoolbook both have slope $\rightarrow 2$ (balanced $\approx 6.7\times$ fewer limb operations at 8,192 digits); balanced-Karatsuba has slope $\rightarrow 1.594$.	EXP-011
GIN-OBS-014	CPython ≥ 3.11 refuses <code>int$\leftrightarrow$decimal-string</code> conversion beyond 4,300 digits by default, because the conversion is quadratic and was a denial-of-service vector (CVE-2020-10735): a runtime system that treats a cost boundary as an admissibility boundary.	EXP-011
GIN-OBS-015	Harmonic sum H_{20000} in binary32: forward 10.480757, backward 10.480734, correctly rounded exact 10.480728 (backward error $\approx 5\times$ smaller).	EXP-012
GIN-OBS-016	$1/x$ in binary64 for $x = 10^{-320}$ (subnormal) gives $+\infty$ with <i>overflow</i> ; for $x = 10^{-330}$ (which rounds to $+0$ when parsed) it gives $+\infty$ with <i>divideByZero</i> .	EXP-003

G. Frontier assessment (honest)

What session 1 has produced, measured against the mandate’s list:

- **A pedagogical and computational framework: yes.** Arithmetic is presented as CGT operational grammars with a four-layer outcome model, an executable SDK, a 20-domain evaluator and cross-validated machine models.
- **A useful unification: partly.** The converse-admissibility reading (GIN-THM-001, COR-001) puts $1/\emptyset, \emptyset/\emptyset, \emptyset - 1$ in $\mathbb{N}$, $7/2$ in $\mathbb{Z}$, $\sqrt{-1}$, modular inverses and CRT into one image/kernel classification. This is standard linear algebra and ring theory; the unification is in the presentation, and its pedagogical value is the claim, not its mathematical novelty. The extension trilemma (GIN-THM-004) organizes known systems by *which law gives way*.
- **New formal definitions: yes, provisional.** Converse admissibility, the repair taxonomy, admissibility closure, the four-layer outcome. Their usefulness beyond exposition is untested.
- **New algorithms, complexity results or lower bounds: no.** Every algorithm and bound here is classical; measured costs match the textbook exponents.
- **New computational representations: no.** (Bijective numeration, 2-adic two’s complement and the projective line are classical.)
- **New insights into machine arithmetic: modest, empirical.** The measured divergence of seven toolchains on one host, the gcc/clang difference on the post-division zero test, the empirical determination of the host’s tininess detection, and the observation that RISC-V’s choice is what a test-free restoring divider computes are useful, reproducible facts; none is a theorem.
- **Genuinely novel mathematical results: none established.** The most likely candidates for statements not found in the reviewed literature are GIN-PROP-006 (heap $\leftrightarrow$ shifted bijective binary) and the pair-level formulation of GIN-THM-006 (every indeterminate form is the null pair); both are elementary and probably folklore. Their novelty is **unverified**, and no claim of priority is made.

H. Reinterpretations

ID	Reinterpretation
GIN-REI-001	Number vs numeral is CGT’s semantics vs syntax: a numeral is an operation expression, its value the denotation at 0.
GIN-REI-002	A canonical numeral is a CGT grammatical address; zero’s address is the empty word.
GIN-REI-003	Inverse operations are converse relations; “undefined” splits into image failure (no solution) and kernel failure (non-unique).
GIN-REI-004	The constructions $\mathbb{N} \rightarrow \mathbb{Z} \rightarrow \mathbb{Q}$ are admissibility repairs that keep the unanswered converse problem as data (GIN-PROP-017).
GIN-REI-005	Carry-lookahead is CGT arithmetic compilation (CGT-DEF-011) of the carry automaton; ripple vs prefix is CGT-THM-004’s shape trade-off.
GIN-REI-006	The exponent’s numeral is a program (GIN-PROP-041); Church numerals are numbers as iterators (GIN-DEF-013).
GIN-REI-007	“Fixed point” has three technically distinct arithmetic senses — an element with $f(x) = x$ (0 and 1 for squaring; idempotents $e^2 = e$ in $\mathbb{Z}/n$), the limit of an iteration (Newton’s method for $\sqrt{a}$; CGT’s PageRank), and the least fixed point of a set operator ($\mathbb{N}$ is the least $X \supseteq \{0\} \cup S(X)$) — unified in definition only.
GIN-REI-008	Positional notation is CGT’s M-SHARE (grammar compression) applied to unary numerals (GIN-PROP-007).
GIN-REI-009	Long division is a finite transition system on remainders; periodic decimals are its cycles (GIN-PROP-009).
GIN-REI-010	The segment notation $a\{I\}b$ is formal notation for a marked path, without new content (§B).

I. Conjecture

GIN-CONJ-001 Re-association as a mechanism (*Conjecture / methodological proposal*)

CGT’s five mechanisms (CGT-DEF-009) should be extended by **M-ASSOC**: when a computation is a product in an associative operation, re-associating a right-linear evaluation into a balanced tree trades work for depth (prefix adders, balanced numeral evaluation, parallel prefix, repeated squaring), and changes *bit* work only together with a cost model that rewards balanced operands (GIN-OBS-013). Status: the examples are classical; whether M-ASSOC is genuinely distinct from CGT-THM-004’s grammar-shape effect, or merely names it, is open (GIN-OPEN-004).

J. Corrections made during session 1

ID	Correction
C-001	The first evaluator treated exponents as elements of $\mathbb{Z}/n$, so 0^{-1} in $\mathbb{Z}/6$ evaluated as $0^5 = 0$. Exponents are now integers acting on the domain (GIN-NEG-013).
C-002	The IEEE simulator first treated all NaNs as quiet; GIN-EXP-005 found 101 flag mismatches, all on signaling-NaN operands. Signaling NaNs are now modelled (GIN-REJ-002).
C-003	The simulator’s docstring first claimed AArch64 detects tininess after rounding; this was unverified and is removed. The x86-64 claim is now measured (GIN-OBS-009).

IV. Machines

Machine Models: how a mathematical grammar becomes an executable one

“The machine” is not one thing. Every row below names its source: **[M]** measured on the session host (x86-64, Linux 6.18; toolchains in `EXPERIMENTS.md`), **[S]** taken from a specification (not executed). Data: `experiments/results/exp004.json`, `exp005.json`.

1. From `x / y` to an instruction

1. **Lexical analysis:** `x`, `/`, `y` become identifier, operator, identifier tokens.
2. **Parsing:** a tree `BinOp(Name x, Div, Name y)` — CPython’s AST is exactly this [M].
3. **Typing:** the operand types select the operation: `int / int` (truncating integer division in C, Java, Go, Rust; true division to float in Python 3), `double / double`, `BigInt / BigInt`.
4. **Intermediate representation and lowering:** CPython compiles to `LOAD_NAME x`; `LOAD_NAME y`; `BINARY_OP /` [M]; C compilers lower to `sdiv/fdiv` IR operations.
5. **Instruction selection** (clang 18 -O2) [M, compile only for non-x86]:

type	x86-64	AArch64	RISC-V 64
int	<code>cltd; idivl</code>	<code>sdiv</code>	<code>divw</code>
unsigned	<code>xorl; divl</code>	<code>udiv</code>	<code>divuw</code>
double	<code>divsd</code>	<code>fdiv</code>	<code>fdiv.d</code>

6. **Execution:** the instruction’s semantics on the operands (below), including traps and flags.

2. Integer division at its two boundaries

model	<code>x / 0</code>	<code>INT_MIN / -1</code>	source
x86-64 <code>idiv</code>	#DE trap → SIGFPE	#DE trap → SIGFPE	[S] Intel SDM; [M] via C
AArch64 <code>sdiv</code>	0, no signal	<code>INT_MIN</code>	[S] Arm ARM (secondary sources)
RISC-V <code>div / divu</code>	$-1 / 2^w - 1$; rem = dividend	<code>INT_MIN</code> ; rem 0	[S] RISC-V ISA, M extension
ISO C	undefined behaviour	undefined behaviour	[S] C17 §6.5.5; [M] gcc: SIGFPE on x86
Java	<code>ArithmeticException</code>	<code>INT_MIN</code> (no exception)	[S] JLS §15.17.2; [M]
Rust (debug and release)	panic “attempt to divide by zero”	panic “attempt to divide with overflow”	[S] Rust reference; [M]
Rust <code>checked_div</code>	None	None	[M]
Go	run-time panic “integer divide by zero”	<code>INT_MIN</code> (no panic)	[S] Go spec; [M]
Python <code>//</code>	<code>ZeroDivisionError</code>	2^{31} (unbounded ints)	[S] Python reference; [M]
JS <code>BigInt</code>	<code>RangeError</code>	2^{31}	[S] ECMA-262; [M]
constant <code>1/0</code> at compile time	Go: compile error; Rust: compile error (deny lint); C: warning, then SIGFPE at run time; Java: compiles, <code>ArithmeticException</code> at run time		[M]

Rounding of integer quotients. Truncation toward zero (x86, C, Java, Go, Rust, JS `BigInt`: $-7/2 = -3$) vs floor (Python: $-7/2 = -4$) [M]. Mathematically both are *selections* from the solutions of $a = qb + r$ with different remainder conditions.

Optimization under undefined behaviour [M]: for `int q = a / b`; if `(b == 0)` return -1; return q; clang 18 -O2/-O3 compiles to `idiv; ret` (test deleted); gcc 13.3 -O2 keeps `test; cmov`. Both are conforming: the test is unreachable in every execution with defined behaviour.

3. Floating point (IEEE 754 binary64) [M] on x86-64 SSE, flags via `fenv.h`

operation	result	flags
<code>1 / 0</code>	inf	<code>divideByZero</code>
<code>-1 / 0</code> , <code>1 / -0</code>	-inf	<code>divideByZero</code>
<code>0 / 0</code>	NaN (printed -nan: the x86 default NaN has its sign bit set)	<code>invalid</code>
<code>0 / 1</code>	0	<code>none</code>
<code>1 / 3</code>	0.333...	<code>inexact</code>
<code>1e308 / 1e-10</code>	inf	<code>overflow, inexact</code>
<code>1e-310 / 1e10</code>	9.99989e-321 (subnormal)	<code>underflow, inexact</code>
<code>√-1</code>	NaN	<code>invalid</code>

Language layers on top: Java, Go, Rust, JavaScript and C return these data; **Python raises `ZeroDivisionError` for 1.0/0.0** [M]. Tininess is detected after rounding on this host [M, GIN-OBS-009]. The reference simulator `ginsdk.ieee` matches results and flags on 80,600 cases [M, GIN-IMP-001].

4. Fixed-width addition overflow

policy	2147483647 + 1 (int32)	where
wrap	−2147483648	hardware add on all three ISAs; Java; Go; Rust release
saturate	2147483647	DSP and SIMD saturating instructions
trap/exception	panic	Rust debug; checked arithmetic
undefined	—	ISO C signed overflow

5. Arbitrary precision

CPython `int`: unbounded; cost grows with operand length (GIN-EXP-008 secondary timings); `int` ↔ decimal string conversion capped at 4,300 digits by default since 3.11 (GIN-OBS-014). `ginsdk.bigint` makes the limb-level cost explicit.

6. Universal claims GiN does not make

GiN does not claim that any behaviour above holds on other hosts, other compiler versions or other flags, except where a specification is cited. AArch64 and RISC-V results are specification-level only (GIN-OPEN-009).

Formal Models

1. Cost models

- **Bit/word model (default for growing operands).** Operands are measured by bit length L . A word operation acts on w -bit limbs ($w = 32$ in `ginsdk.bigint`): add/subtract with carry, limb product, two-by-one-limb quotient estimate, limb comparison (GIN-DEF-070). Statements of the form “ $\Theta(L^2)$ ” refer to counted word operations.
- **Unit-cost model (bounded operands only).** One arithmetic operation costs 1 when operands fit in a fixed width; the width is stated (e.g. `int32`). Every unit-cost claim names the bound — this is the arithmetic form of CGT-THM-003 (“ $O(1)$ only while the address fits in a word”).
- **Rule-application model.** For Peano arithmetic, one rewrite step per primitive-recursion equation (GIN-DEF-010).
- **Circuit model.** Gate count and depth over a stated basis with fan-in ≤ 2 (GIN-DEF-040). Fan-out and wire length are not modelled (Brent & Kung’s area model is not used).
- **Wall-clock time** is secondary evidence only, recorded in fields ending `_s`.

2. The four layers of an arithmetic outcome

GIN-DEF-061 refines CGT’s three layers by inserting **typing** between syntax and semantics:

Layer	Question	Formal object	Example where it fails
syntax	well-formed sentence?	parse tree	<code>1 +</code>
typing	do literals and variables denote elements of the domain?	lifted operands	<code>0.5</code> in $\mathbb{Z}$; <code>2147483648</code> in <code>int32</code> ; <code>x</code> with no value
semantics	does each operation have exactly one result?	Sol (GIN-DEF-015)	<code>1/0</code> (no solution), <code>0/0</code> (non-unique), <code>0 - 1</code> in $\mathbb{N}$
execution	what does the machine model do?	outcome (GIN-DEF-062)	<code>2147483647 + 1</code> wraps or is UB; <code>1/0</code> traps on x86; <code>1.0/0.0</code> raises in Python

The layers are independent in the sense of CGT-PROP-002: syntactically fine but ill-typed (`0.5` in $\mathbb{Z}$); well-typed but semantically empty (`1/0` in $\mathbb{Q}$); semantically fine but failing at execution (`((-2147483647 - 1)/-1)` has the unique integer solution `2,147,483,648`, but x86 traps); semantically inadmissible but executing to a value (`1/0` on AArch64 gives `0`).

3. The expression language

Grammar (GIN-DEF-060), precedence low $\rightarrow$ high:

```

expr  := term (('+' | '-' ) term)*
term  := unary (('*' | '/' | '/' | '%' | 'mod') unary)*
unary := ('+' | '-' ) unary-at-30 | power
power := atom ('^' unary)?           (right-associative; -2^2 = -(2^2))
atom  := NUMBER | 'x' | '(' expr ')' | ('sqrt' | 'gcd') '(' expr ',' expr '*' ')'
NUMBER := decimal integer | decimal fraction | 0b... | ...2 | 0x...

```

Unicode `×` `÷` `−` are accepted as aliases. Exponents are evaluated in exact $\mathbb{Z}$ regardless of the domain (exponentiation is an action, GIN-NEG-013).

4. Domains of evaluation (`ginsdk.expr.DOMAINS`)

Exact: $\mathbb{N}$, $\mathbb{Z}$, $\mathbb{Q}$, $\mathbb{R}$ (rational arithmetic plus flagged 40-digit approximations of irrational roots), $\mathbb{C}$ (Gaussian rationals), $\mathbb{Z}/n$. Floating point: `binary64`, `binary32`, `binary16`, `binary8` (a 1-4-3 teaching format), “JS Number” (= `binary64`). Languages and ISAs (32-bit): `x86-64`, `AArch64`, `RISC-V`, `C`, `Java`, `Rust` (debug), `Go`; `Python` (unbounded int, true division to `binary64`, exceptions on zero divisors); `JS BigInt`. Symbolic: rational functions of x over $\mathbb{Q}$ with definedness conditions.

5. Cost accounting conventions in the evaluator

Exact domains report a schoolbook bit-operation *estimate* ($\max(L_a, L_b) + 1$ for $\pm$, $L_a - L_b$ for $\times$ and $\div$) — a cost model, not a measurement; floating-point and machine domains report one operation per step. The limb-level measurements of GIN-EXP-008 are the authoritative cost data.

V. Number Theory

Number Theory as Transition Systems

Only after arithmetic has been developed structurally does GiN turn to number theory. For each topic: the grammar (states and moves), the invariant, what is preserved, what is expensive, and which representation makes it efficient. Results: RESULTS.md §E; experiments GIN-EXP-009, 010.

Topic	States	Legal moves	Invariant / preserved	Expensive	Efficient representation
divisibility $b \mid a$	—	the converse problem $bc = a$ (image condition)	—	as costly as one division (GIN-OPEN-002)	any; mod m is finite-state MSB-first
gcd (Euclid)	(a, b)	$(a, b) \rightarrow (b, a \bmod b)$ if $b \neq 0$	$\gcd(a, b)$	$\Theta(L^2)$ bit ops; $\leq \log_{\varphi} a$ steps (Lamé)	positional; binary gcd uses shifts
Bézout	rows (r, s, t)	$r_{i+1} = r_{i-1} - q_i r_i$ (same for s, t)	$r = sa + tb$	coefficient growth bounded by $b/(2g)$	—
lcm	—	$\text{lcm} = ab /\gcd$	—	one gcd	—
modular arithmetic $\mathbb{Z}/n$	residues	$+, -, \times \bmod n$; / when admissible	the residue class	division needs a gcd	residues $0 \dots n-1$
congruences, CRT	residue tuples	converse of $x \mapsto (x \bmod m_i)$	—	pairwise gcds	mixed radix
prime factorization	multisets of primes	split a composite	the product	best known methods are super-polynomial in L	factor lists
fundamental theorem of arithmetic	—	—	uniqueness of factorization in $\mathbb{Z}$ (a unique-normal-form property of the multiplicative grammar)	—	—
primality: trial division	candidate divisors	$d \leftarrow d + 2$ while $d^2 \leq n$	—	$\approx \sqrt{n}/2$ divisions (exponential in L)	—
primality: sieve	a bit array	cross out multiples	—	$\sim n \ln \ln n$ crossings	precomputation (CGT M-PRE)
Fermat / Miller-Rabin	witnesses a	modular exponentiation	—	$O(L^3)$ bit ops per base, schoolbook	exponent program (GIN-PROP-041)
modular exponentiation	accumulator	S (square) / SM (square, multiply) per exponent bit	$\text{acc} = x^{\text{prefix of } e}$	$\lceil \log_2 e \rceil$ squarings, popcount $- 1$ multiplications	binary (or windowed) exponent
Pollard rho	(x, y)	$x \leftarrow f(x), y \leftarrow f(f(y))$	—	$\sim \sqrt{p}$ steps for the smallest factor p	Floyd cycle detection
arithmetic functions φ, μ, σ	—	from the factorization	multiplicativity	factoring	factor lists
recurrence sequences (Fibonacci)	(F_k, F_{k+1})	$(x, y) \rightarrow (y, x + y)$	—	the worst case of Euclid (Lamé)	matrix powers by the exponent program
Diophantine $ax + by = c$	—	converse of $(x, y) \mapsto ax + by$	—	one extended gcd	solvable iff $\gcd(a, b) \mid c$; solutions form a coset of the kernel $\{(bt/g, -at/g)\}$ — GIN-THM-001 again

Counterexamples

Fermat pseudoprimes (341 base 2), Carmichael numbers (561; 43 below 10^6), strong pseudoprimes (2047 base 2; 46 below 10^6): admissibility tests that admit composites (GIN-NEG-002).

Does CGT reveal useful structural decompositions?

Three, none of them new algorithms: (1) the **image/kernel** reading unifies division, modular inverses, linear Diophantine equations and the CRT (GIN-THM-001, PROP-042); (2) the **exponent program** shows square-and-multiply, addition chains and SLP compression of unary numerals as one object (GIN-PROP-007, 041); (3) the **finite transition system on remainders** explains periodic expansions and divisibility automata (GIN-PROP-009, THM-005). No improvement over classical algorithms was found or claimed.

VI. Evidence

Experiments

Code: experiments/ (git history is the version record). **Reproduce:** `sh experiments/run_all.sh` ($\approx$ 1.5 minutes; writes experiments/results/*.json and results/SUMMARY.md). **Determinism:** fixed seeds; `python3 experiments/verify_reproduction.py` re-runs everything and reports identical counted fields (last run: identical in all files). **Environment of the recorded runs:** CPython 3.13.16, Linux 6.18 x86_64, Intel Xeon @ 2.80 GHz; gcc 13.3.0, clang 18.1.3, OpenJDK 21.0.12.1, rustc 1.97.0, go 1.24.7, Node.js 22.22.0 (recorded per file in environment). **Cost methodology:** primary dependent variables are counted steps under the cost model stated in each script's docstring (rule applications, bit flips, gates and depth, 32-bit limb operations, division steps). Wall-clock time is secondary and appears only in fields ending in `_s`. **Theorem checks:** `experiments/check_theorems.py` checks the exact statement of every “proved here” result on small cases (21 checks, all passing). They guard against mis-statement; they are not proofs. **What is measured vs specified:** machine experiments (EXP-004, EXP-005) run on the host. AArch64 and RISC-V behaviour is *not* measured (no such hardware or emulator was available); those models are specification models and are labelled as such.

GIN-EXP-001 Numerals and the successor grammar (`exp001_numerals.py`)

- **Hypotheses.** H-001 ... H-005.
- **Design.** (1) For bases 2, 3, 10, enumerate all words up to length L and count those denoting $n < 30$. (2) For bijective bases 1, 2, 3, 10, enumerate all words up to a length bound and check that values are distinct and form an initial segment. (3) Heap conjugacy for $n < 200,000$. (4) Two's complement vs 2-adic truncation for widths 4, 8, 16. (5) Peano rule counts vs binary bit operations for $n = 2^k - 1$, $k \leq 20$.
- **Falsification.** Any count off the formula $L - |\text{addr}(n)| + 1$; any collision in a bijective system; any heap mismatch.
- **Result.** All formulas hold; 0 collisions; 0 mismatches (PROP-001...006, 008; OBS-001).

GIN-EXP-002 Increment and carries (`exp002_increment_carries.py`)

- **Hypotheses.** H-006, H-007. **Seed** 20261008.
- **Design.** Exact flips for every increment up to 2^{17} ; longest carry chain over 3,000 ($n \leq 1024$) or 1,000 random pairs for $n = 4 \dots 4096$; K/P/G frequencies.
- **Result.** Flip formula exact; chains $\log_2 n - 0.66 \dots 0.84$ (OBS-002); K, P, G $\approx 1/4, 1/2, 1/4$.
- **Limitation.** The constant offset depends on the definition of chain length; GiN counts the generating position plus the propagating positions it travels through.

GIN-EXP-003 Converse admissibility census (`exp003_converse_census.py`)

- **Hypotheses.** H-008 ... H-010. Exhaustive; no randomness.
- **Design.** All (a, b) in $\mathbb{Z}/n$ for $n \leq 64$; the meaning table of 14 expressions $\times$ 15 domains via `ginsdk.expr`; the $x/0$, $0/x$, x/x families exactly and symbolically; $1/x$ in binary64 as $x \rightarrow 0^\pm$; other converse problems ($\sqrt{}$, $\log$, modular inverse).
- **Result.** 0 coset violations; unique share = $\varphi(n)/n$; the meaning table (SUMMARY); OBS-003, OBS-016.

GIN-EXP-004 Machine division on this host (`exp004_machine_division.py`, sources in `experiments/machine/`)

- **Hypotheses.** H-011 ... H-013, H-037.
- **Design.** C compiled with gcc -O0 and -O2 (operands from argv to prevent constant folding): int $x/0$, $0/0$, $\text{INT_MIN}/-1$, $-7/2$; double $x/0$, $0/0$, signed zero, overflow, underflow, $\sqrt{-1}$, with fenv flags. Post-division zero test compiled by gcc -O0/-O2 and clang -O2/-O3, assembly inspected. Constant $1/\emptyset$ in C, Go, Rust, Java (compile and run). Java, Rust (debug and release; `/`, `checked_div`, `wrapping_div`), Go, JavaScript (Number and BigInt), Python (in process; AST and bytecode of `x / y`). Instruction selection with clang -O2 for x86_64, aarch64 and riscv64 targets (compile only).
- **Result.** OBS-004...008; model agreement 14/14 (IMP-002).
- **Limitation.** One host; one version per toolchain; AArch64/RISC-V not executed.

GIN-EXP-005 IEEE simulator vs hardware (`exp005_ieee_validation.py`, oracle `machine/fpharness.c`)

- **Hypotheses.** H-014, H-015. **Seed** 754.
- **Design.** For binary32 and binary64: 40,000 cases over $+$, $-$, $\times$, $\div$, $\sqrt{}$ (random bit patterns; special and boundary values; nearby exponents; subnormal territory) plus 300 targeted products whose exact value lies in $(\lambda(1 - 2^{-p}), \lambda)$, λ = smallest normal — the only region where before- and after-rounding tininess differ. Compare results (NaN-ness only for NaNs) and the five flags, under both tininess models.
- **Result.** 0 result mismatches; 0 flag mismatches with after-rounding tininess (after modelling signaling NaNs); 158/159 mismatches with before-rounding, all in the targeted family (IMP-001, OBS-009).
- **History.** The first run showed 92 (binary32) + 9 (binary64) flag mismatches, all signaling-NaN operands (REJ-002); fixed in `ginsdk.ieee`, then re-run.

GIN-EXP-006 Adder circuits ([exp006_adder_circuits.py](#))

- **Hypotheses.** H-016, H-017. **Seed** 6.
- **Design.** Ripple-carry, Sklansky, Kogge–Stone, Brent–Kung; exhaustive $n \leq 8$ (65,536 pairs at $n = 8$), 300 random pairs for $n = 16, 32, 64$; gates and depth for $n = 1 \dots 256$; carry monoid table; NAND-only full adder.
- **Result.** 0 errors; ripple $5n - 3 / 2n - 1$; at $n = 256$: Sklansky 3,584 / 18, Kogge–Stone 5,891 / 17, Brent–Kung 2,018 / 30, ripple 1,277 / 511.

GIN-EXP-007 Multiplier and divider circuits ([exp007_mul_div_circuits.py](#))

- **Hypotheses.** H-018, H-019.
- **Design.** Array multiplier and restoring divider; exhaustive $n \leq 6$; zero-divisor outputs compared with the RISC-V DIVU/REMU and AArch64 UDIV models; sizes for $n \leq 32$.
- **Result.** 0 errors; zero divisor gives $(2^n - 1, a) = \text{RISC-V}$ for all $n \leq 6$ (PROP-023).

GIN-EXP-008 Bit complexity ([exp008_bit_complexity.py](#))

- **Hypotheses.** H-020, H-021. **Seed** 8.
- **Design.** Random L-bit operands, $L = 64 \dots 16,384$ (gcd to 4,096, modexp to 512): counted limb operations of `ginsdk.bigint`; CPython timings as secondary data; log-log slopes between successive sizes.
- **Result.** OBS-010.

GIN-EXP-009 Euclid ([exp009_euclid.py](#))

- **Hypotheses.** H-022 ... H-024. **Seed** 9.
- **Design.** All pairs below 1,200 (smallest pair per step count, histogram); average steps for primes $n = 1009 \dots 1,000,003$; Bézout coefficient bounds on 20,000 random pairs; binary vs Euclidean gcd on 5,000 random 64-bit pairs.
- **Result.** Lamé pairs match; OBS-011; 0 bound violations; binary gcd ≈ 45.2 subtractions vs Euclid ≈ 37.8 divisions (each subtraction is cheaper).

GIN-EXP-010 Primes ([exp010_primes.py](#))

- **Hypotheses.** H-025 ... H-028. **Seed** 10.
- **Design.** Sieve to 10^6 with crossing count; Fermat base-2 and strong base-2 pseudoprimes and Carmichael numbers below 10^6 ; deterministic Miller–Rabin vs sieve; trial-division counts for primes of 11 ... 31 bits; Pollard rho on $p \cdot q$ with p of 10 ... 26 bits.
- **Result.** 245 / 46 / 43 (published values); 0 disagreements; OBS-012.

GIN-EXP-011 Numeral evaluation shapes ([exp011_numeral_shapes.py](#))

- **Hypotheses.** H-029, H-030. **Seed** 11.
- **Design.** Decimal numerals of $k = 64 \dots 8,192$ digits converted to binary limbs by Horner, balanced composition with schoolbook multiplication, and balanced composition with Karatsuba; counted limb operations; compositions and depth.
- **Result.** OBS-013, OBS-014.

GIN-EXP-012 Representation boundaries ([exp012_representation_boundaries.py](#))

- **Hypotheses.** H-031 ... H-035.
- **Design.** Least unrepresentable integer by search (full search for $p = 4, 11$; windows for $p = 24, 53$); exactly representable k -digit decimals, $k \leq 4$; associativity over all tenth-triples; harmonic sum H_{20000} in binary32 in both orders; $\text{MAX} + 1$ for widths 8 ... 64 under each overflow policy.
- **Result.** PROP-030, PROP-031, PROP-032, OBS-015.

Experimental summary (generated by experiments/analyze.py)

Every number below is read from experiments/results/*.json. Counted fields are deterministic; wall-clock fields (suffix _s) are not reported here.

GIN-EXP-001 Numerals and the successor grammar

- Non-injectivity = leading zeros, formula $L - |addr(n)| + 1$ words: standard base 2: holds, standard base 3: holds, standard base 10: holds.
- Bijective numeration: bijective base 1: 21 words, 0 collisions, initial segment = True; bijective base 2: 32767 words, 0 collisions, initial segment = True; bijective base 3: 29524 words, 0 collisions, initial segment = True; bijective base 10: 111111 words, 0 collisions, initial segment = True.
- Heap conjugacy (bijective base 2 $\leftrightarrow$ CGT heap, shifted by 1): 0 mismatches for $n < 200000$.
- Two's complement = truncated 2-adic expansion: 0 mismatches (widths 4, 8, 16).

bits k	$n = 2^k - 1$	Peano add rules	binary add bit ops	Peano mul rules	schoolbook mul bit ops
1	1	2	2	4	1
4	15	16	5	256	16
7	127	128	8	16384	49
10	1023	1024	11	1048576	100
13	8191	8192	14	—	169
16	65535	65536	17	—	256
19	524287	524288	20	—	361

GIN-EXP-002 Increment and carries

- Counting $0 \rightarrow N$ flips exactly $2N - \text{popcount}(N)$ bits: 0 mismatches for $N \leq 131072$; amortized 2.0000 flips per increment; worst single increment 18.

n	mean longest carry chain	$\log_2 n$	difference	max
4	1.1997	2.0	-0.8003	4
8	2.1593	3.0	-0.8407	8
16	3.191	4.0	-0.809	12
32	4.3013	5.0	-0.6987	21
64	5.3063	6.0	-0.6937	15
128	6.2847	7.0	-0.7153	19
256	7.3433	8.0	-0.6567	20
512	8.2923	9.0	-0.7077	18
1024	9.2993	10.0	-0.7007	20
2048	10.261	11.0	-0.739	21
4096	11.271	12.0	-0.729	20

GIN-EXP-003 Converse admissibility census

- Z/n , $n \leq 64$, all pairs: coset-size violations 0; unique share = $\varphi(n)/n$: True; solvable pairs = $\sum n/\text{gcd}(b, n)$: True.

Meaning table (status in each domain):

expression	N	Z	Q	C	Z/6	Z/7	binary64	Python	JS BigInt	int32 x86-64	int32 AArch64
6 / 3	2	2	2	2	non-unique	2	2	2.0	2	2	2
0 / 1	0	0	0	0	0	0	0	0.0	0	0	0
1 / 0	no-solution	no-solution	no-solution	no-solution	no-solution	no-solution	Infinity	exception	exception	trap	0
0 / 0	non-unique	non-unique	non-unique	non-unique	non-unique	non-unique	NaN	exception	exception	trap	0
-1 / 0	no-solution	no-solution	no-solution	no-solution	no-solution	no-solution	-Infinity	exception	exception	trap	0
1 / -0	no-solution	no-solution	no-solution	no-solution	no-solution	no-solution	-Infinity	exception	exception	trap	0
7 / 2	no-solution	no-solution	7/2	7/2	no-solution	0	3.5	3.5	3	3	3
0 - 1	no-solution	-1	-1	-1	5	6	-1	-1	-1	-1	-1
2 / 4	no-solution	no-solution	1/2	1/2	non-unique	4	0.5	0.5	0	0	0
sqrt(-1)	no-solution	no-solution	no-solution	i	unsupported	unsupported	NaN	unsupported	unsupported	unsupported	unsupport
0^0	1	1	1	1	1	1	unsupported	1	1	unsupported	unsupport
0^-1	no-solution	no-solution	no-solution	no-solution	no-solution	no-solution	unsupported	exception	exception	unsupported	unsupport
2147483647 + 1	2147483648	2147483648	2147483648	2147483648	2	2	2147483648	2147483648	2147483648	-2147483648	-2147483648

expression	N	Z	Q	C	Z/6	Z/7	binary64	Python	JS BigInt	int32 x86-64	int32 AArch64
(-2147483647 - 1) / -1	no-solution	2147483648	2147483648	2147483648	2	2	2147483648	2147483648.0	2147483648	trap	-2147483648

GIN-EXP-004 Machine division on this host

Toolchains: gcc: gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0; clang: Ubuntu clang version 18.1.3 (1ubuntu1); java: openjdk version "21.0.12.1" 2026-08-18; rustc: rustc 1.97.0 (2d8144b78 2026-07-07); go: go version go1.24.7 linux/amd64; node: v22.22.0.

behaviour	measured
C (gcc -O0) int 1/0	SIGFPE
C (gcc -O0) int INT_MIN/-1	SIGFPE
C (gcc -O0) double 1/0	inf flags=[divideByZero]
C (gcc -O0) double 0/0	-nan flags=[invalid]
C (gcc -O0) double 1/-0	-inf flags=[divideByZero]
constant 1/0 — C (gcc -Wall)	exit 0: machine/const_div.c:2:35: warning: division by zero [-Wdiv-by-zero]
constant 1/0 — C (gcc) at run time	exit -8, SIGFPE:
constant 1/0 — Go	exit 1: machine/const_div.go:5:31: invalid operation: division by zero
constant 1/0 — Rust	exit 1: error: this operation will panic at runtime
constant 1/0 — Java (javac)	exit 0:
constant 1/0 — Java at run time	exit 1: Exception in thread "main" java.lang.ArithmeticException: / by zero
if (b == 0) after a / b — gcc -O0	zero test kept
if (b == 0) after a / b — gcc -O2	zero test kept
if (b == 0) after a / b — clang -O2	zero test deleted
if (b == 0) after a / b — clang -O3	zero test deleted
java int 1/0	ArithmeticException: / by zero
java int MIN/-1	-2147483648
java double 1/0	Infinity
java double -1/0	-Infinity
rust_debug div 1/0	thread 'main' panicked at machine/divide.rs:7:33:
rust_debug div MIN/-1	thread 'main' panicked at machine/divide.rs:7:33:
rust_debug checked 1/0	None
rust_debug checked MIN/-1	None
rust_debug wrapping MIN/-1	-2147483648
rust_debug f64 1/0	inf
rust_release div 1/0	thread 'main' panicked at machine/divide.rs:7:33:
rust_release div MIN/-1	thread 'main' panicked at machine/divide.rs:7:33:
rust_release checked 1/0	None
rust_release checked MIN/-1	None
rust_release wrapping MIN/-1	-2147483648
rust_release f64 1/0	inf
go int32 1/0	panic: runtime error: integer divide by zero
go int32 MIN/-1	-2147483648
go float64 1/0	+Inf
JavaScript 1/0	Infinity
JavaScript -1/0	-Infinity
JavaScript 1/-0	-Infinity
JavaScript 0/0	NaN
JavaScript 0/1	0
JavaScript 6/3	2
JavaScript 7/2	3.5
JavaScript `(2**31)	0`
JavaScript 0.1+0.2	0.30000000000000004
JavaScript 2**53+1	9007199254740992
JavaScript 1n/0n	RangeError: Division by zero
JavaScript 0n/0n	RangeError: Division by zero
JavaScript 7n/2n	3
JavaScript -7n/2n	-3
JavaScript `(-(2n**31n))/-1n	2147483648

Instruction selection (clang -O2): x86_64: idiv → idivl, udiv → divl, fdiv → divsd; aarch64: idiv → sdiv, udiv → udiv, fdiv → fdiv; riscv64: idiv → divw, udiv → divuw, fdiv → fdiv.d.

Model agreement with measurement: x86 model: 1/0 traps: yes, x86 model: MIN/-1 traps: yes, x86 model: -7/2 = -3: yes, Java model: MIN/-1 = MIN: yes, Java model: 1/0 exception: yes, Rust model: 1/0 panics: yes, Rust model: MIN/-1 panics (debug): yes, Rust

model: MIN/-1 panics (release): yes, Go model: 1/0 panics: yes, Go model: MIN/-1 = MIN: yes, Python model: -7//2 = -4: yes, Python model: 1//0 raises: yes, JS BigInt model: 1n/0n RangeError: yes, JS BigInt model: -7n/2n = -3: yes.

GIN-EXP-005 IEEE simulator versus hardware

- binary32: 40300 cases; tininess-after model: 0 result and 0 flag mismatches; tininess-before model: 158 flag mismatches (all in the targeted family, where hardware raised underflow in 142/300).
- binary64: 40300 cases; tininess-after model: 0 result and 0 flag mismatches; tininess-before model: 159 flag mismatches (all in the targeted family, where hardware raised underflow in 141/300).
- Conclusion: this host detects tininess **after rounding**.

GIN-EXP-006 Adder circuits

Exhaustive verification $n \leq 8$ and random $n \in \{16, 32, 64\}$: 0 errors.

n	ripple gates / depth	Sklansky	Kogge-Stone	Brent-Kung
1	2 / 1	2 / 1	2 / 1	2 / 1
2	7 / 3	7 / 3	7 / 3	7 / 3
4	17 / 7	20 / 6	23 / 5	20 / 6
8	37 / 15	52 / 8	67 / 7	49 / 10
16	77 / 31	128 / 10	179 / 9	110 / 14
32	157 / 63	304 / 12	451 / 11	235 / 18
64	317 / 127	704 / 14	1091 / 13	488 / 22
128	637 / 255	1600 / 16	2563 / 15	997 / 26
256	1277 / 511	3584 / 18	5891 / 17	2018 / 30

GIN-EXP-007 Multiplier and divider circuits

- Exhaustive errors ($n \leq 6$): 0.
- Zero divisor: restoring divider output equals RISC-V DIVU/REMU ($q = 2^n - 1$, $r = \text{dividend}$) for every $n \leq 6$: True.

n	multiplier gates / depth	divider gates / depth
1	1 / 1	16 / 9
2	8 / 3	50 / 19
4	64 / 16	172 / 51
8	320 / 40	632 / 163
16	1408 / 88	2416 / 579
32	5888 / 184	9440 / 2179

GIN-EXP-008 Bit complexity (32-bit limb operations)

bits	add	mul (school)	mul (Karatsuba)	divmod 2L/L	gcd Euclid	powmod
64	2	8	8	16	130	2320
128	4	32	32	50	476	14483
256	8	128	128	155	1582	109761
512	16	512	512	568	6085	828405
1024	32	2048	1796	2167	22000	—
2048	64	8192	5838	8399	81034	—
4096	128	32768	18409	33200	326516	—
8192	256	131072	56771	131928	—	—
16384	512	524288	172598	526069	—	—

Log-log slopes between successive sizes: add: 1.0 (last); sub: 1.0 (last); compare_equal: 1.0 (last); mul_schoolbook: 2.0 (last); mul_karatsuba: 1.604 (last); divmod_2L_by_L: 1.996 (last); gcd_euclid: 2.011 (last); gcd_binary: 1.952 (last); powmod_L_bit_exponent: 2.916 (last).

GIN-EXP-009 Euclid

- Lamé: smallest pair for each step count is a consecutive Fibonacci pair: True (pairs below 1200).
- Average steps for prime n versus $(12 \ln 2/\pi^2) \ln n + \text{Porter's constant}$: $n = 1009$: 6.287698 vs 7.29625 (diff -1.008551); $n = 10007$: 8.230762 vs 9.229829 (diff -0.999067); $n = 100003$: 10.170387 vs 11.169805 (diff -0.999418); $n = 1000003$: 12.109263 vs 13.110322 (diff -1.001059). The difference is -1 up to $\mathcal{O}(1)$: Porter's τ_n counts one extra initial step ($m \bmod n$ with $m < n$).
- Bézout coefficient bounds: 0 violations in 20000 samples.

GIN-EXP-010 Primes

- Sieve to 10^6 : 78498 primes, 2122048 crossings ($0.8082 \times n \ln \ln n$).
- Below 10^6 : 245 base-2 Fermat pseudoprimes, 46 strong base-2 pseudoprimes, 43 Carmichael numbers (published: {‘fermat_base2’: 245, ‘strong_base2’: 46, ‘carmichael’: 43}).
- Deterministic Miller–Rabin vs sieve: 0 disagreements.
- Pollard rho mean steps / $\sqrt{p}$: 10-bit p: 0.911, 14-bit p: 0.966, 18-bit p: 1.134, 22-bit p: 1.052, 26-bit p: 0.908.

GIN-EXP-011 Numeral evaluation shapes

digits	Horner	balanced (school)	balanced (Karatsuba)	depth balanced
64	705	408	408	6
128	2598	1023	1023	7
256	10207	2856	2856	8
512	40339	8704	8388	9
1024	159519	29177	24659	10
2048	633291	104333	73694	11
4096	2529644	390586	221901	12
8192	10105494	1507887	669698	13

Last log-log slopes: horner: 1.998, balanced_schoolbook: 1.949, balanced_karatsuba: 1.594.

GIN-EXP-012 Representation boundaries

- Least unrepresentable positive integer: binary8 (1-4-3 teaching format): 17 ($= 2^p + 1$: True); binary16: 2049 ($= 2^p + 1$: True); binary32: 16777217 ($= 2^p + 1$: True); binary64: 9007199254740993 ($= 2^p + 1$: True).
- Exactly representable k-digit decimals in binary64: k = 1: 1 of 9, k = 2: 3 of 99, k = 3: 7 of 999, k = 4: 15 of 9999.
- Associativity of binary64 addition over tenths: 242 of 729 triples fail; commutativity failures: 0.
- Harmonic sum H_{20000} in binary32: forward 10.480757, backward 10.480734, correctly rounded exact 10.480728.

Theorem checks (check_theorems.py)

identifier	result	scope
GIN-PROP-001	ok	6303 word pairs
GIN-PROP-002	ok	bases 2...16
GIN-PROP-003	ok	m, n < 12
GIN-PROP-004	ok	bijective bases 1,2,3,5 exhaustive by length; residue algorithm on 3 grammars
GIN-PROP-005	ok	base 7,
GIN-PROP-006	ok	n < 300000
GIN-PROP-008	ok	widths 1...12, all representable n
GIN-PROP-009	ok	bases 2,3,10,12, denominators < 200
GIN-PROP-010	ok	$N \leq 65536$
GIN-THM-001	ok	30 rings, 5737 pairs
GIN-THM-002	ok	$n \leq 60$
GIN-THM-003	ok	$\mathbb{Z}/n$, n < 60
GIN-PROP-015	ok	four formats, both signs
GIN-PROP-021	ok	n < 200
GIN-PROP-023	ok	$n \leq 6$, all dividends
GIN-PROP-030	ok	full search for p = 4, 11; boundary windows for p = 24, 53
GIN-PROP-031	ok	$k \leq 5$
GIN-PROP-040	ok	all pairs below 700, step counts 1...13
GIN-PROP-041	ok	e < 5000; the bit string of e is recovered from the move word
GIN-PROP-042	ok	moduli < 16

Hypothesis Ledger

Status values: **open**, **supported** (evidence agrees, no counterevidence, not proved), **established** (proved or exhaustively verified within a stated scope; see the result), **partly supported**, **rejected**, **reformulated** (superseded by a sharper statement).

GIN-H-000 The founding question (*A. Kadaan*)

- **Statement.** Arithmetic and elementary number theory can be described as computational grammars: objects, symbols, states, admissible operations, transitions, invariants, information requirements and machinery, such that “undefined” expressions are explained by which grammatical condition fails.
- **Status. Reformulated** into H-001 ... H-035. The descriptive half is supported throughout: every operation studied has a CGT operational grammar, and every failure studied is classified by layer and cause (GIN-DEF-061, GIN-THM-001, GIN-THM-004). Whether the description yields new *mathematics* is answered “not so far” (RESULTS §G).

ID	Statement (short)	Test	Status	Evidence
GIN-H-001	The value map on standard numerals is surjective and non-injective exactly by leading zeros.	EXP-001	established	PROP-001
GIN-H-002	Zero’s canonical numeral is the empty word.	EXP-001	established	PROP-002
GIN-H-003	Bijjective notation removes all non-injectivity.	EXP-001	established	PROP-004
GIN-H-004	The CGT heap grammar is bijective binary, shifted by one.	EXP-001	established	PROP-006
GIN-H-005	Unary (Peano) arithmetic costs time linear in the values, i.e. exponential in binary length.	EXP-001	established	PROP-003, OBS-001
GIN-H-006	Counting $0 \rightarrow N$ flips exactly $2N - \text{popcount}(N)$ bits.	EXP-002	established	PROP-010
GIN-H-007	The mean longest carry chain is $\log_2 n + O(1)$.	EXP-002	supported (measured $n \leq 4096$; known in the literature)	OBS-002
GIN-H-008	Division failures split into exactly two kinds, and in a commutative ring the solution set of $b \cdot c = a$ is empty or a coset of $\text{Ann}(b)$.	EXP-003	established	THM-001
GIN-H-009	In $\mathbb{Z}/n$ the share of pairs with a unique quotient is $\varphi(n)/n$.	EXP-003	established	THM-002
GIN-H-010	The same expression fails at different layers in different domains.	EXP-003, EXP-004	supported	OBS-003, OBS-004
GIN-H-011	“Division by zero” is not one behaviour on real systems.	EXP-004	supported (seven toolchains, one host)	OBS-004...006
GIN-H-012	The SDK’s machine models agree with every measurable behaviour on the host.	EXP-004	supported (14/14)	IMP-002
GIN-H-013	Instruction selection maps / to different instructions per ISA and type.	EXP-004	supported	OBS-007
GIN-H-014	Exact rational evaluation + one RNE rounding reproduces hardware results bit for bit.	EXP-005	supported (80,600 cases)	IMP-001
GIN-H-015	...and reproduces all five flags when tininess is detected as on the host.	EXP-005	supported after correction (signaling NaNs had to be modelled)	IMP-001, REJ-002
GIN-H-016	Ripple: $5n - 3$ gates, depth $2n - 1$; prefix adders: more gates, logarithmic depth.	EXP-006	established (ripple), supported (prefix: measured $n \leq 256$)	PROP-021, OBS in SUMMARY
GIN-H-017	All four adders are correct.	EXP-006	established $n \leq 8$, supported to $n = 64$	IMP-003
GIN-H-018	A restoring divider without a divisor test outputs $(2^n - 1, \text{dividend})$ on a zero divisor = RISC-V DIVU/REMU.	EXP-007	established	PROP-023
GIN-H-019	Array multiplier and divider sizes grow quadratically.	EXP-007	supported ($n \leq 32$)	SUMMARY
GIN-H-020	Counted word operations: linear (+, −, cmp), quadratic (schoolbook $\times$, $\div$, gcd), $n^{1.585}$ (Karatsuba), cubic (modexp).	EXP-008	supported	OBS-010
GIN-H-021	CPython int timings show the same ordering.	EXP-008	supported (secondary evidence only)	exp008.json
GIN-H-022	Lamé: smallest pair with k steps is $(F_{\lfloor k/2 \rfloor}, F_{\lfloor k/2 \rfloor + 1})$.	EXP-009	established (classical; checked)	PROP-040
GIN-H-023	Average steps follow $(12 \ln 2/n^2) \ln n + \text{Porter’s constant}$.	EXP-009	supported with a convention offset of exactly 1	OBS-011
GIN-H-024	Extended-Euclid coefficients are bounded by $b/(2g)$, $a/(2g)$.	EXP-009	supported (20,000 samples)	exp009.json
GIN-H-025	Published pseudoprime counts below 10^6 (245 / 46 / 43).	EXP-010	supported (reproduced exactly)	PROP-043
GIN-H-026	Deterministic Miller-Rabin agrees with the sieve up to 10^6 .	EXP-010	supported	PROP-043
GIN-H-027	Sieve $\sim n \ln \ln n$ crossings; trial division $\sim \sqrt{n}/2$ divisions for primes.	EXP-010	supported (sieve ratio 0.81 at 10^6 — the lower-order terms are not negligible there)	exp010.json
GIN-H-028	Pollard rho $\sim \sqrt{p}$ steps.	EXP-010	supported	OBS-012

ID	Statement (short)	Test	Status	Evidence
GIN-H-029	Horner and balanced numeral evaluation use the same compositions; depth k vs $\lceil \log_2 k \rceil$.	EXP-011	established	PROP-005
GIN-H-030	Shape alone gains no bit-cost exponent; it pays asymptotically only with a subquadratic multiplication.	EXP-011	supported (and refined: shape alone gives a constant factor ≈ 6.7 at 8,192 digits)	OBS-013, REJ-003
GIN-H-031	Least unrepresentable positive integer is $2^p + 1$.	EXP-012	established	PROP-030
GIN-H-032	Exactly $2^k - 1$ of the k -digit decimals are binary-representable.	EXP-012	established	PROP-031
GIN-H-033	Float + is commutative but not associative.	EXP-012	established	PROP-032
GIN-H-034	Summation order changes the computed harmonic sum.	EXP-012	supported	OBS-015
GIN-H-035	MAX + 1 gives MIN under wraparound for every width.	EXP-012	established (definition of wraparound)	exp012.json
GIN-H-036	Every indeterminate form of projective arithmetic on pairs is the null pair $(0, 0)$, the unique pair that cannot be an element.	check_theorems	established	THM-006
GIN-H-037	Optimizing compilers delete a zero test that follows a division by the tested variable.	EXP-004	partly supported : clang 18 does, gcc 13.3 does not	OBS-008, NEG-004

Counterexamples and Negative Results

Each entry is a constructed or measured case in which a plausible claim about arithmetic, or a claim made earlier in this investigation, fails. These cases set the scope of the theory.

ID	Plausible claim	Counterexample	Failure mode	Source
GIN-NEG-001	Floating-point addition is associative.	$(0.1 + 0.1) + 0.4 = 0.6000000000000001 \neq 0.6 = 0.1 + (0.1 + 0.4)$ in binary64; 242 of 729 tenth-triples fail.	one rounding per operation	PROP-032, EXP-012
GIN-NEG-002	If $a^{n-1} \equiv 1 \pmod{n}$ then n is prime.	$341 = 11 \cdot 31$ (base 2); $561 = 3 \cdot 11 \cdot 17$ passes every coprime base; 245 base-2 pseudoprimes below 10^6 .	Fermat's condition is necessary, not sufficient	PROP-043, EXP-010
GIN-NEG-003	Division by zero is always an error.	AArch64 SDIV returns 0 and RISC-V DIV returns -1 , with no signal; IEEE returns $\pm\infty$.	totalization (repair 5)	THM-004, MACHINE_MODEL
GIN-NEG-004	Compilers always exploit “division by zero is undefined” to delete a later zero test.	gcc 13.3 -O2 keeps the test (test + cmove); clang 18 -O2 deletes it.	the permission is not an obligation; behaviour is compiler-specific	OBS-008
GIN-NEG-005	$1/0 = \infty$.	In IEEE 754, $0 \times \infty = \text{NaN}$, so ∞ does not solve $0 \cdot c = 1$; on the projective line ∞ exists but $0 \cdot \infty$ is undefined; in $\mathbb{R}$ there is no ∞ .	the value is a limit-motivated selection, not a converse solution	PROP-015, THM-006
GIN-NEG-006	Symbolic simplification preserves meaning.	$x/x \rightarrow 1$ and $(x^2 - 1)/(x - 1) \rightarrow x + 1$ enlarge the domain (to include 0, resp. 1).	cancellation drops the condition $g \neq 0$	PROP-014
GIN-NEG-007	Truncated subtraction is subtraction.	$0 \div 1 = 0$ but $1 \div 0 \neq 0$.	totalization, not converse	PROP-016
GIN-NEG-008	Re-shaping an evaluation (balanced instead of Horner) makes it asymptotically cheaper.	With schoolbook multiplication both shapes are $\Theta(k^2)$ in limb operations (slopes 1.998 vs 1.949 $\rightarrow$ 2).	shape changes depth; bit work depends on the multiplication used	OBS-013
GIN-NEG-009	0^0 has a determined value.	As the empty product $0^0 = 1$ by convention; as a limit form $\lim f^g$ with $f, g \rightarrow 0$ it is indeterminate. The evaluator marks the convention.	convention vs converse/limit	expr, EXP-003
GIN-NEG-010	Python floats follow IEEE 754 on division by zero.	Python raises <code>ZeroDivisionError</code> for $1.0/0.0$ (language check before the IEEE operation); C, Java, Go and JavaScript return $+\text{inf}$.	the language adds a signal layer	OBS-006
GIN-NEG-011	<code>-2147483648</code> is an <code>int32</code> literal.	In C (and in GiN's evaluator) it is unary minus applied to <code>2147483648</code> , which does not fit in <code>int32</code> ; libraries define <code>INT_MIN</code> as $(-2147483647 - 1)$.	typing layer	expr, EXP-003
GIN-NEG-012	Brahmagupta's $0/0 = 0$ is inconsistent.	It is a consistent <i>selection</i> : meadows and Lean's mathlib adopt exactly $0^{-1} = 0$. It fails only as a claim to <i>solve</i> $0 \cdot c = 0$ uniquely.	conflating totalization with derivation	HIST-001
GIN-NEG-013	(own design error) Exponents can be evaluated in the same domain as the base.	0^{-1} in $\mathbb{Z}/6$ evaluated as $0^5 = 0$, because $-1 \equiv 5$. Exponents are integers acting on the domain (and only exponents $\equiv \text{mod}$ the group exponent may be reduced, for units).	type confusion between a ring and the integers acting on it	RESULTS §J C-001
GIN-NEG-014	A complete residue digit system gives unique numerals.	$b = 2$, $D = \{-1, 2\}$; the word $(-1)(2)$ denotes 0, so leading “zero-denoting” blocks are not just zeros.	prefixes denoting 0	PROP-004
GIN-NEG-015	Ordinary binary names every integer.	The residue expansion of -1 never terminates (...111); fixed-width hardware truncates it.	negative numbers have only infinite (2-adic) standard numerals	PROP-008
GIN-NEG-016	Adjoining a “ $0/0$ ” element to the fractions is harmless.	$(0,0)$ is cross-multiplication-related to every pair, so $(1,0) \sim (0,0) \sim (0,1)$ while $(1,0) \not\sim (0,1)$: transitivity fails.	the null pair collapses the quotient	THM-006
GIN-NEG-017	Floating-point overflow and division by zero are the same event.	$1/10^{-320} \rightarrow +\infty$ with overflow (the divisor is subnormal, not zero); $1/10^{-330} \rightarrow +\infty$ with <code>divideByZero</code> (the literal rounded to $+0$).	distinct flags, same datum	OBS-016

Rejected hypotheses

- **GIN-REJ-001** “ $1/0 = +\infty$ answers the converse question.” Rejected by PROP-015 and THM-006 (NEG-005).
- **GIN-REJ-002** (session-1 working assumption) “The simulator need not model signaling NaNs to match hardware flags.” Rejected by EXP-005: all 101 initial flag mismatches were operations on signaling NaNs, where hardware raises *invalid*. Kept as evidence that the experiment can find defects in the instrument.
- **GIN-REJ-003** “Balanced evaluation of numerals is asymptotically cheaper than Horner's rule in bit cost.” Rejected for schoolbook multiplication (NEG-008); the surviving form needs a subquadratic multiplication.
- **GIN-REJ-004** “Compilers delete post-division zero tests” as a general statement (NEG-004).

VII. CGT and the Frontier

CGT Concept Map: how Computational Grammar Theory behaves on numbers

This document records, concept by concept, what happens when CGT’s machinery is applied to the structure $\mathbb{N}$ (and its relatives $\mathbb{Z}$, $\mathbb{Q}$, $\mathbb{R}$, $\mathbb{C}$, $\mathbb{Z}/n$, fixed-width words, floating-point formats). Each row says whether the concept **survives unchanged**, **needs refinement**, or **becomes especially clear**, and what arithmetic reveals that CGT had not isolated.

Source: the CGT ledger at commit `b3a29f3` (read-only; see `PROVENANCE.md` §3). Verdicts are judgements of this investigation, not theorems.

1. Summary table

CGT concept	Arithmetic instance	Verdict	Where in GiN
Computational structure vs representation (CGT-DEF-001)	number vs numeral vs encoding vs machine state	especially clear ; refined into four layers	GIN-DEF-001...004
Move signature, relational semantics (CGT-DEF-002)	S, P ; digit moves $x \mapsto bx + d$; $\cdot +b$, $\cdot \times b$	survives; refined : binary operations give move <i>families</i> indexed by an operand (an infinite signature)	GIN-DEF-005, GIN-DEF-014
Operational grammar (CGT-DEF-003)	numerals = words over digit moves; admissibility language = canonical numerals	survives unchanged	GIN-DEF-005...007
Three layers syntax/semantics/execution (CGT FORMAL_MODELS §2, CGT-PROP-002)	$1/\emptyset$ well formed, semantically empty; $\emptyset/\emptyset$ semantically non-functional; IEEE executes both	refined into four layers: a <i>typing</i> layer is needed ($\emptyset.5$ in $\mathbb{Z}$, the literal <code>2147483648</code> in <code>int32</code>)	GIN-DEF-061
Semantic realizability $\llbracket w \rrbracket(x) \neq \emptyset$	“no solution”	survives; refined : arithmetic needs a name for	$\llbracket w \rrbracket(x)$
(absent in CGT) converse moves	division = converse of multiplication; subtraction = converse of addition; $\sqrt{}$ = converse of squaring	new to the framework : inverse operations are relational converses; their admissibility is functionality + totality at a point	GIN-DEF-015...017, GIN-THM-001
Structural grammar (CGT-DEF-004)	$\mathbb{N} \rightarrow 0 \mid S(\mathbb{N})$; positional numerals as straight-line programs for unary terms	especially clear : positional notation <i>is</i> grammar compression of unary numerals (CGT’s M-SHARE)	GIN-REI-008
Cost profile, word RAM (CGT-DEF-005, FORMAL_MODELS §1)	unit-cost arithmetic vs bit cost	refined : the growing parameter is operand bit length; unit cost is admissible only for bounded operands	GIN-DEF-070, GIN-EXP-008
Address, numeration, density (CGT-DEF-006)	canonical numeral = shortlex-least word reaching n from 0	especially clear : positional notation is the address grammar; zero’s address is the empty word; CGT’s heap numeration is bijective binary shifted by one	GIN-PROP-002, GIN-PROP-006
Transition monoid (CGT-DEF-007)	digit grammar $\rightarrow$ affine maps $x \mapsto b^*x + c$; carry automaton $\rightarrow \{K, P, G\}$	survives; two instructive cases (infinite vs finite monoid)	GIN-PROP-005, GIN-PROP-020
Arithmetic compilability (CGT-DEF-011)	carry words compile to one of 3 elements; digit words compile to (b^k, c) of $\Theta(k)$ bits	refined : “ $O(1)$ words” fails for digit words (the compiled object grows); holds for the carry monoid. The finite/infinite transition monoid is the regular/non-regular boundary	GIN-REI-005, GIN-THM-005
Information content (CGT-DEF-008)	k -bit numerals name 2^k values; $2^k - 1$ of the k -digit decimals are binary-representable	survives	GIN-PROP-031
Mechanisms M-ARITH, M-PRE, M-SHARE, M-RESTRICT, M-PRUNE (CGT-DEF-009)	see §3	all five appear; a sixth candidate, re-association (M-ASSOC), is proposed	GIN-CONJ-001
Arithmetic addressing scope (CGT-THM-003)	n -bit numerals cost $\lceil n/w \rceil$ words; <code>MAX_INT + 1</code>	survives; becomes the fixed-width boundary	GIN-EXP-012
Same semantics, different grammar shape (CGT-THM-004)	ripple vs prefix adders; Horner vs balanced numeral evaluation	especially clear , with a caveat: shape changes depth; it changes bit work only with a matching cost model	GIN-EXP-006, GIN-EXP-011
Grammars cannot cross P/NP (CGT-PROP-006)	factoring, discrete log	survives; GiN makes no complexity-class claims	§4
PageRank fixed points (CGT-THM-008)	$f(x) = x$; idempotents; Newton iteration; $\mathbb{N}$ as least fixed point	partially unifies; important distinctions remain	GIN-REI-007
Counterexample discipline, ladder of claims	—	survives unchanged; extended with HISTORICAL RESULT, REINTERPRETATION, IMPLEMENTATION RESULT	RESEARCH_CHARTER.md §3

2. Concept by concept

2.1 Structure and representation $\rightarrow$ four layers

CGT separates a structure from its representations (CGT-DEF-001) and warns that many apparent “grammar advantages” are changes of representation (CGT-OBS-010). Arithmetic is the paradigm case. The value 2 (an element of $\mathbb{N}$), the numerals `2`, `102`, `II`, `55` (words), the encodings `0x00000002` (`int32`) and `0x4000000000000000` (`binary64`), and a register holding one of these encodings in some machine state are four different objects. GiN makes them four layers (GIN-DEF-001...004) with explicit maps between them:

denotation (numeral $\rightarrow$ value), *encoding* (value $\rightarrow$ bit string, partial), *decoding* (bit string $\rightarrow$ value, total on finite data), *execution* (state $\rightarrow$ state). $1 + 1 = 2$ and $1 + 1 = 10_2$ state the same equation at the value layer; they differ at the numeral layer (Chapter 5).

2.2 Moves and move families

CGT’s move signature Σ is finite. The successor S is a single move; but “add b ” and “multiply by b ” are *families* of moves indexed by b . Arithmetic therefore needs either an infinite signature or a two-argument operation viewed as a relation on pairs. GiN uses both views: digit grammars have finite signatures (one move per digit); the arithmetic expression language treats binary operators as relations $[\circ] \subseteq (A \times A) \times A$.

2.3 Converse moves — what arithmetic adds

CGT relations may be non-deterministic, and semantic emptiness $\llbracket w \rrbracket(x) = \emptyset$ is a recognized phenomenon (CGT-PROP-002 case 1). What CGT did not isolate is the systematic role of **converses**. Every inverse operation of arithmetic is the converse of a deterministic forward move:

$$a - b = c \Leftrightarrow b + c = a, a / b = c \Leftrightarrow b \cdot c = a, \sqrt{a} = c \Leftrightarrow c \cdot c = a \text{ (and } c \geq 0), \log a = c \Leftrightarrow e^c = a.$$

The converse of a function is in general a *relation*, neither total nor functional. An inverse expression is admissible at a point exactly when the converse relation is a function there: one and only one output. This gives the trichotomy no solution / unique / non-unique (GIN-DEF-016), which classifies $1/0$ and $0/0$ as *different* failures, as the research mandate anticipated, and puts them in one family with $0 - 1$ in $\mathbb{N}$, $7/2$ in $\mathbb{Z}$, $\sqrt{-1}$ in $\mathbb{R}$, $\sqrt{4}$ before a branch is chosen, and 2^{-1} in $\mathbb{Z}/6$ (GIN-THM-001...003).

2.4 Structural grammar and compression

$\mathbb{N}$ is generated by the structural grammar $\mathbb{N} \rightarrow 0 \mid S(\mathbb{N})$: the smallest non-trivial structural grammar. The unary term $S^n(0)$ has size n . A binary numeral of n is a straight-line program of size $O(\log n)$ that generates the same term, with productions corresponding to “double” ($X \mapsto XX$) and “double then succeed”. This is exactly CGT’s **M-SHARE** mechanism, and it is exponential compression: the unary term of 2^k has size 2^k , the binary numeral size $k + 1$. The smallest SLP for the unary word a^n corresponds to a shortest addition chain for n (Charikar et al. 2005; see LITERATURE_REVIEW.md §5), which is also the problem of computing x^n with fewest multiplications. Positional notation, square-and-multiply exponentiation, and grammar compression of unary words are one grammar seen three ways (GIN-REI-008).

2.5 Addresses

CGT-DEF-006 defines the address of x as the shortlex-least word w with $\llbracket w \rrbracket(\text{root}) = x$. For the digit grammar of base b rooted at 0 this is the ordinary base- b numeral without leading zeros (GIN-PROP-002). Two consequences:

1. The address of 0 is the **empty word**. The written numeral “0” is a non-canonical word (one leading zero). Positional notation needs a visible zero digit for two different reasons: as a placeholder *inside* numerals (**101**) and as a conventional non-empty spelling of the empty address.
2. The CGT heap numeration of binary trees (node 1 is the root, children $2x$ and $2x + 1$) is conjugate, via $x \mapsto x + 1$, to *bijective* base-2 notation (digits 1, 2) rooted at 0 (GIN-PROP-006). The founding CGT observation that “an address carries operations” is, for numbers, the observation that a numeral is a program.

2.6 Transition monoids and compilation

For a digit grammar the transition monoid is a monoid of affine maps $x \mapsto b^k x + c$; compiling a word gives (b^k, value) and yields the concatenation law (GIN-PROP-005). The compiled element is not $O(1)$ machine words: c has $\Theta(k)$ bits. So digit grammars are **not** arithmetically compilable in the CGT-DEF-011 sense for unbounded words. By contrast, the carry automaton of binary addition has a **finite** transition monoid $\{K, P, G\}$ (kill, propagate, generate), so any word of digit pairs compiles into one of three elements, and associativity yields parallel-prefix (carry-lookahead) adders (GIN-PROP-020). The difference between a finite and an infinite transition monoid is the difference between addition, which a finite automaton can perform on numerals, and multiplication, which it cannot (GIN-THM-005).

2.7 Mechanisms (CGT-DEF-009)

Mechanism	Arithmetic instance
M-ARITH	positional notation itself: the address grammar computes instead of storing (heap = bijective binary)
M-PRE	multiplication tables; precomputed powers in windowed exponentiation; sieves as precomputed admissibility (primality) tables
M-SHARE	positional notation as SLP compression of unary; repeated squaring
M-RESTRICT	restricting divisors to units ($\mathbb{Z}/n$), to nonzero elements (fields), operands to bounded width (fixed-width arithmetic is unit cost <i>because</i> of the restriction)
M-PRUNE	admissibility checks before execution: Go’s and Rust’s compile-time rejection of constant $1/0$, <code>checked_div</code> , <code>b $\neq$ 0</code> guards
M-ASSOC (proposed)	associativity lets a right-linear evaluation be re-associated into a balanced tree: carry-lookahead adders, balanced numeral evaluation, parallel prefix. CGT-THM-004 already exhibits the effect; GiN proposes naming it as a sixth mechanism (GIN-CONJ-001).

2.8 Grammar shape (CGT-THM-004)

Ripple-carry (right-linear) and prefix adders (balanced) compute the same function; measured costs: ripple $5n - 3$ gates and depth $2n - 1$, Kogge-Stone 5891 gates and depth 17 at $n = 256$ (GIN-EXP-006). For numeral evaluation the two shapes perform the same

number of compositions, but in *bit cost* the balanced shape is asymptotically better only when multiplication is subquadratic (GIN-EXP-011). Shape is a mechanism only relative to a cost model — a refinement of the CGT lesson.

2.9 Fixed points

“Fixed point” appears in arithmetic in at least three technically different senses: (i) an element with $f(x) = x$ (0 and 1 for squaring, idempotents in $\mathbb{Z}/n$); (ii) the limit of an iteration (Newton’s method for $\sqrt{a}$; CGT’s PageRank, CGT-THM-008), where existence, uniqueness and rate matter; (iii) the *least* fixed point of a set operator ($\mathbb{N}$ is the least X with $\{0\} \cup S(X) \subseteq X$, the inductive definition behind recursion and induction). One definition covers all three formally; the computational content does not transfer between them (GIN-REI-007).

3. What arithmetic reveals that CGT had not isolated

1. **Converse admissibility** as a first-class notion (§2.3): CGT treated relational semantics generally but did not single out inverse operations or classify their failures.
2. **A typing layer** between syntax and semantics (§2.1).
3. **Bit length as the cost parameter** (§1, row “cost profile”).
4. **Finite vs infinite transition monoids** as the regular/non-regular boundary of operations on numerals (§2.6).
5. **Re-association** as a candidate mechanism (§2.7).
6. **Machine semantics are plural**: the same expression has seven different meanings on seven toolchains on one host (GIN-EXP-004). CGT’s “execution layer” must name the machine.

4. What does not transfer

- CGT’s PageRank and AI-systems material has no arithmetic counterpart beyond fixed-point iteration.
- CGT-PROP-006 (no P/NP crossing) has no arithmetic analogue that GiN can add to: GiN makes no claim about the complexity of factoring or discrete logarithms.
- CGT’s GaaS (Grammar as a Service) contract is not reproduced; the GiN expression evaluator returns a comparable layered outcome object but is not a service.

5. Proposed extensions to CGT (for the CGT researcher to consider; not applied)

- E1. Add *converse admissibility* (GIN-DEF-015...017) to CGT’s operational grammars, with the no-solution/unique/non-unique classification.
- E2. Add a typing layer to CGT’s three-layer model.
- E3. Consider M-ASSOC as a sixth mechanism.
- E4. Cite GIN-PROP-006 (heap = shifted bijective binary) next to CGT-DEF-006.

Open Problems

Ordered roughly by how informative a solution would be for the program.

GIN-OPEN-001 A grammar-class hierarchy of arithmetic operations

GIN-THM-005 separates operations a finite automaton can perform on numerals (successor, addition, comparison, multiplication and division by constants, reduction modulo constants) from those it cannot (multiplication). Place the remaining operations of this book — gcd, modular exponentiation with variable modulus, integer square root, base conversion — in a precise hierarchy (synchronous rational, rational, pushdown, linear-time, ...) and relate the class to bit complexity. Much is known (Cobham 1969; Büchi; automatic structures); the task is to state the hierarchy for these operations with references and fill any gaps.

GIN-OPEN-002 The cost of admissibility

Deciding whether a/b is admissible in $\mathbb{Z}$ means deciding $b \mid a$, which costs as much as dividing. In $\mathbb{Z}/n$, deciding whether b is a unit costs a gcd. When is deciding admissibility asymptotically cheaper than computing the result, and when is it as hard? Is there a natural converse problem where admissibility is hard but computation, when admissible, is easy?

GIN-OPEN-003 Repairs as universal constructions

GIN-DEF-017 lists seven repairs. Several are universal constructions: the Grothendieck group (extension for $+$), localization (division by a multiplicative set), completion, algebraic closure, quotient. Characterize which repairs are left adjoints, which are not (totalization, signalling), and whether the extension trilemma (GIN-THM-004) has a categorical statement.

GIN-OPEN-004 Is re-association a distinct mechanism?

Make GIN-CONJ-001 precise: define when a speedup is “attributable to re-association” and decide whether M-ASSOC is distinct from CGT-THM-004’s grammar-shape effect, or a special case of it (see CGT-OPEN-001, CGT-OPEN-007).

GIN-OPEN-005 Literature debts

Primary sources not yet read in full: Carlström 2004 (exact wheel axioms), Bergstra & Tucker 2007, Knuth 1978 (carry propagation), Sorenson & Webster 2017 (ψ_{13} value taken from secondary sources), the ARM Architecture Reference Manual SDIV pseudocode, Smullyan 1961 and Forslund 1995 (bijective numeration). Search specifically for prior statements of GIN-PROP-006 and of the pair-level form of GIN-THM-006.

GIN-OPEN-006 Completeness of the four-layer outcome model

Does syntax · typing · semantics · execution capture the failure modes of other numeric systems: decimal floating point, posits/unums, interval arithmetic, arbitrary-precision libraries with rounding modes, exact real arithmetic, computer-algebra systems? Find a failure that does not fit.

GIN-OPEN-007 A complete IEEE 754 reference

Extend `ginsdk.ieee` to all rounding-direction attributes, fused multiply-add, conversions, remainder, decimal formats, NaN payloads; validate against an established test suite and against more than one ISA (an AArch64 host would test before-rounding tininess).

GIN-OPEN-008 Does the segment notation generalize?

GIN-REI-010 shows $a\{I\}b$ is notation for a marked path. For a transition system in general, is there a useful “boundary/interior” notation for intervals between two states (posets, CGT trees, the Stern-Brocot tree of rationals) that supports a calculus (composition of segments, boundary operators) rather than display only?

GIN-OPEN-009 Machine semantics on more hosts

Measure GIN-EXP-004 on AArch64 and RISC-V hardware (or faithful emulators), on Windows, and for more compilers and versions; in particular, map which compilers delete post-division zero tests and under which flags (GIN-OBS-008).

GIN-OPEN-010 Admissibility closures beyond $\mathbb{Q}$

The closure of $\mathbb{Q}$ in $\mathbb{R}$ under field operations and square roots of positive elements is the field of real constructible numbers; under all polynomial converse problems it is the real algebraic numbers. Develop GIN-DEF-019 into a precise correspondence between sets of converse problems and classical closures, and determine which closures are computable with which bit complexity.

GIN-OPEN-011 Mechanized proofs

Formalize GIN-THM-001...006 and GIN-PROP-006 in a proof assistant. Note that Lean’s `mathlib` uses the totalization $0^{-1} = 0$, so the formal statements must be phrased with explicit admissibility hypotheses — itself a test of the converse vocabulary.

Literature Review (session 1 baseline)

The canonical bibliography, with per-entry verification status, is `book/content/bibliography.json` (57 entries: 30 metadata-verified by web search on 2026-10-08, 12 cited from standards or secondary sources, 15 from the collaborator’s knowledge and marked unverified). “**Metadata verified**” never means the full text was read. Primary-source reading is a recorded debt (GIN-OPEN-005).

For each area: what is known, what GiN adopts, and what (if anything) GiN adds.

1. Foundations of number

- **Known.** Dedekind (1888) and Peano (1889) characterize $\mathbb{N}$ as the successor algebra; primitive recursion defines $+$, $\times$, $\wedge$; Church (1941) represents numbers as iterators. The integers, rationals, reals and complex numbers are obtained by standard constructions (Grothendieck group, field of fractions, completion, algebraic closure).
- **GiN adopts** all of this unchanged (GIN-DEF-001, 009, 010, 013).
- **GiN adds** a reading of the constructions $\mathbb{N} \rightarrow \mathbb{Z} \rightarrow \mathbb{Q}$ as *admissibility repairs* that keep the unanswered converse problem as data (GIN-PROP-017), and the observation (GIN-THM-006) that in the pair construction the null pair $(0, 0)$ is exactly the obstruction to adjoining “0/0”. Both are presentations of classical mathematics.

2. Positional notation, bijective numeration, b-adic expansions

- **Known.** Positional systems, radix conversion and their costs (Knuth 1997, §4.1, §4.4); bijective base- k (“ k -adic”) notation (Smullyan 1961; Forslund 1995); balanced ternary; b -adic expansions and two’s complement as truncated 2-adic integers (folklore; Knuth §4.1).
- **GiN adds** the CGT packaging: numerals as operation expressions with affine moves, canonical numerals as CGT grammatical addresses with zero’s address the empty word (GIN-PROP-002), and the conjugacy of CGT’s heap numeration with shifted bijective binary (GIN-PROP-006; not found in the sources reviewed, likely folklore).

3. Automata and arithmetic

- **Known.** Büchi (1960) connected weak monadic second-order arithmetic with finite automata; Cobham (1969) showed recognizability depends on the base; Hodgson (1983) and Khoussainov & Nerode (1995) introduced automatic structures, whose first-order theories are decidable; Blumensath & Grädel (2000) developed their theory. $(\mathbb{N}, +)$ is automatic; $(\mathbb{N}, +, \times)$ is not, because its theory is undecidable (Gödel 1931; Church 1936).
- **GiN adopts** these results as GIN-THM-005 and reads them as a boundary between regular and non-regular operational grammars over numerals. The finite carry monoid is the algebraic face of the automaticity of addition (GIN-PROP-020).

4. Carry propagation and adders

- **Known.** The average longest carry chain is about $\log_2 n$ (Burks, Goldstine & von Neumann 1946; Knuth 1978). Carry computation is a prefix computation (Ladner & Fischer 1980; Brent & Kung 1982), with classical networks by Sklansky (1960) and Kogge & Stone (1973). Bletloch (1990) surveys prefix sums for any associative operator.
- **GiN adds** the CGT reading: carry-lookahead is CGT arithmetic compilation (CGT-DEF-011) of the carry automaton’s three-element transition monoid; ripple vs prefix is CGT-THM-004’s shape trade-off; and a candidate sixth mechanism (M-ASSOC, GIN-CONJ-001).

5. Grammar compression and addition chains

- **Known.** Straight-line programs; the smallest grammar problem (Charikar et al. 2005). For unary strings, SLPs correspond to addition chains, the classical problem of computing x^n with fewest multiplications (Knuth 1997, §4.6.3).
- **GiN adds** the observation that positional notation is exactly this compression of the successor grammar, connecting CGT’s M-SHARE with exponentiation (GIN-PROP-007, GIN-REI-008).

6. Bit complexity of arithmetic

- **Known.** Schoolbook $\Theta(n^2)$ multiplication and division, Karatsuba & Ofman (1962) $\Theta(n^{\log_2 3})$, and $O(n \log n)$ multiplication (Harvey & van der Hoeven 2021); Brent & Zimmermann (2010) is a modern reference. Euclid’s algorithm takes $O(\log a)$ steps (Lamé 1844) and $\Theta(n^2)$ bit operations naively; its average step count is $(12 \ln 2/\pi^2) \ln n + O(1)$ (Heilbronn 1969; Dixon 1970; Porter 1975). Binary gcd: Stein (1967).
- **GiN adopts** these as cost targets and *measures* them with counted limb operations (GIN-EXP-008, 009, 011). Nothing new.

7. Division by zero and its extensions

- **Known.** Fields exclude division by zero; the projective line and Riemann sphere adjoin a single ∞ ; the extended reals adjoin $\pm\infty$; IEEE 754 (2019) specifies $\pm\infty$, NaN, signed zeros and the divideByZero/invalid flags (Goldberg 1991; Kahan 1987 on the sign of zero). Meadows (Bergstra & Tucker 2007) totalize inversion with $0^{-1} = 0$ while keeping the commutative-ring axioms; Lean’s mathlib uses the same convention (mathlib Community 2020). Wheels (Carlström 2004) extend any commutative ring with an involution “/” so that division by 0 yields elements, at the cost of $0 \cdot x = 0$ and with a weakened distributive law. Historically, Brahmagupta (628 CE) gave rules for zero including $0/0 = 0$, and Bhāskara II named a quantity with zero divisor *khahara* (MacTutor; Plofker 2009).
- **GiN adds** a single classification — image failure (1/0) vs kernel failure (0/0) — that covers fields and all commutative rings (GIN-THM-001, COR-001), the extension trilemma that sorts every known system by *which law gives way* (GIN-THM-004), and the pair-level characterization of indeterminate forms (GIN-THM-006). The pair construction with (0, 0) kept as an absorbing element is essentially Carlström’s wheel of fractions; GiN’s statement that every indeterminate form computes the null pair should be checked against Carlström’s paper before any novelty is suggested.

8. Machine arithmetic semantics

- **Specifications.** Intel SDM (IDIV raises #DE on zero divisor and quotient overflow); Arm ARM (AArch64 SDIV/UDIV: no trap, result 0; secondary sources); RISC-V M extension ($x/0 \rightarrow$ all ones with remainder = dividend; overflow $\rightarrow -2^{\{w-1\}}$, remainder 0; rationale: simple divider circuitry, no trap so languages can add a branch); ISO C17 §6.5.5 (undefined behaviour); JLS §15.17.2 (ArithmeticException; $\text{MIN}/-1 = \text{MIN}$); Rust (panic on zero divisor and on overflow); Go (run-time panic on zero divisor; constant division by zero is a compile error; $\text{MIN}/-1 = \text{MIN}$); ECMAScript (Number follows IEEE; BigInt division by 0n throws RangeError); Python (ZeroDivisionError for int and float).
- **GiN adds** measurements on one host (GIN-EXP-004) confirming each specified behaviour that can be run there, and two empirical observations: the gcc/clang divergence on a post-division zero test (GIN-OBS-008) and the host’s after-rounding tininess detection (GIN-OBS-009).

9. Number theory

- **Known.** Euclid (*Elements* VII); Bézout; the Chinese remainder theorem; Fermat’s little theorem; Korselt’s criterion (1899) and the infinitude of Carmichael numbers (Alford, Granville & Pomerance 1994); Miller (1976) and Rabin (1980) strong probable-prime tests; deterministic base sets (Sorenson & Webster 2017); pseudoprime counts (Pomerance, Selfridge & Wagstaff 1980); Pollard’s rho (1975).
- **GiN adopts** all of these; presents each algorithm as a transition system with an invariant; reproduces the published counts below 10^6 (GIN-EXP-010). Nothing new.

10. Computational Grammar Theory (parent corpus)

- The CGT ledger (Kadaan 2026; read-only) supplies operational grammars, addresses, transition monoids, arithmetic compilation, the mechanism taxonomy and the three-layer separation. `CGT_CONCEPT_MAP.md` records how each behaves on numbers.

11. Areas not yet reviewed

Computer algebra systems’ handling of $x/0$ and of side conditions in simplification; interval arithmetic and its division by intervals containing 0; posits/unums; decimal floating point; proof-assistant conventions beyond Lean’s mathlib; the didactics literature on division by zero; category-theoretic treatments of totalization.

Book Outline

Title: *Grammar in Numbers* **Subtitle:** *Arithmetic, Number Theory, and the Computational Grammar of Number Founding*

Principal Researcher: Abed Kadaan **Relationship:** a specialized frontier volume of the Computational Grammar Theory program; independently buildable; links to CGT but does not depend on it.

Alternative subtitles considered: *What 0, 1, 2, 3 Compute*; *The Admissibility of Arithmetic*; *Arithmetic as a Computational Language*. The working subtitle is kept because it names all three subjects.

How the research changed the mandate’s outline

The mandate’s fourteen-part outline was a good starting point. The evidence of session 1 changed it in four ways:

1. **“Numerals as programs” became its own part (II)**, between “What is a number?” and arithmetic. The digit-grammar view (numerals = operation expressions, addresses, compilation, compression) is what connects GiN to CGT most precisely, and it is needed before $1 + 1 = 10_2$ can be explained.
2. **“Division as a converse” is the spine of Parts III-IV**. The image/kernel theorem (GIN-THM-001) organizes subtraction, division, roots, logarithms, modular inverses and the CRT; division by zero is its most important instance rather than a separate topic.
3. **“Extending the grammar” gets a chapter (10)** of its own, built around the extension trilemma (GIN-THM-004) and the pair theorem (GIN-THM-006).
4. **Mandate Parts IX (number systems), X (number theory) and XI (algorithms) were merged** into Parts VI-VII, because the research found the algorithms to be the clearest presentation of the number theory as transition systems.

Structure (as built in `book/content/book.json`)

Part I — What Is a Number?

1. Number, Numeral, Digit, Machine — the four layers.
2. The Smallest Grammar: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4$ — successor, segments, $\emptyset\{1,2,3\}4$.
3. Recursion, Induction and the Least Fixed Point — Peano, primitive recursion with counted cost, Church numerals, three senses of fixed point.

Part II — Numerals as Programs

4. Positional Notation as an Operational Grammar — digit grammars, canonical numerals, zero’s empty address, bijective notation, the CGT heap.
5. $1 + 1 = 10_2$: Value and Representation — binary, carries, the cost of the successor, two’s complement as 2-adic, periodic expansions.

Part III — Arithmetic as Transition

6. Addition and Subtraction — five readings of $1 + 1$; $0 - 1$ and the construction of $\mathbb{Z}$.
7. Multiplication — 2×3 vs 3×2 ; why $\times 0$ gives 0 (four derivations); numerals as straight-line programs; why multiplication is not finite-state.
8. Division as a Converse — $6 / 3$ six ways; the image/kernel theorem; division in $\mathbb{Z}/n$.

Part IV — The Boundary of Zero

9. The Grammar of Division by Zero — $1/0$ vs $0/0$ vs $0/1$; the $x/0$, $0/x$ table; symbolic vs numeric; limits.
10. Extending the Grammar: What Gives Way — no ring inverts 0; the trilemma; fractions as pairs and the null pair; IEEE, meadows, wheels; Brahmagupta.
11. The Grammar of Zero and One — zero vs NULL, NaN, empty, false, undefined; one as identity and generator.

Part V — Machines That Compute Arithmetic

12. From Symbol to Instruction — lexing to `idiv`; seven toolchains measured; undefined behaviour and the optimizer.
13. Finite Width and Floating Point — overflow policies; IEEE 754; flags; representation boundaries; a simulator checked against hardware.
14. Arithmetic Circuits — gates, adders, the carry monoid, prefix adders, multiplier, divider, and what a divider does with 0.

Part VI — Number Systems as Grammars

15. $\mathbb{N}$, $\mathbb{Z}$, $\mathbb{Q}$, $\mathbb{R}$, $\mathbb{C}$, $\mathbb{Z}/n$, $\mathbb{Z}_p$ — closure, identities, inverses, order, representations; repairs and admissibility closures.

Part VII — Number Theory and Its Algorithms

16. Divisibility and Euclid — the Euclid system, Lamé, Bézout, average steps.
17. Congruences and the Chinese Remainder Theorem — $\mathbb{Z}/n$, inverses, CRT and linear Diophantine equations as converse problems.
18. Exponentiation and Primality — the exponent program; Fermat and its counterexamples; Miller-Rabin; sieves; Pollard rho.

Part VIII — The Cost of Arithmetic

19. Bit Complexity — unit cost vs bit cost; measured exponents; numeral evaluation shapes; the CPython conversion limit.

Part IX — Arithmetic, Computation and CGT

20. What Is the Computational Grammar of Number? — the answer of session 1; the CGT concept map; the honest frontier.
21. Open Problems.

Appendices

A. Notation and conventions. B. Reproducibility, the SDK and the laboratory.

The laboratory (website)

- **Arithmetic Lab** — expression $\rightarrow$ tokens, parse tree, layered outcome, converse equation, steps, representation, cost, in any of 20+ domains; validated against SDK fixtures.
- **Machine Lab** — one expression across all domains, side by side.
- **Zero Explorer** — the line $y = b \cdot c$ against $y = a$: one intersection, none, or all; and the $\mathbb{Z}/n$ multiplication grid.
- **Successor & Numeral Lab** — $a\{I\}b$ parser, transition diagram, numerals of n in unary, bijective, binary, decimal, balanced ternary; compilation.
- **Carry Lab** — K/P/G per position, carries, ripple vs prefix composition.
- **Circuit Lab** — toggleable full adder and 4-bit ripple adder.
- **Float Lab** — IEEE fields, exact value, neighbours, ulp for binary16/32/64.
- **Pairs Lab** — projective arithmetic on pairs; the null pair.
- **Euclid and Exponent Labs** — traces with invariants.

Decision Log

ID	Date	Decision	Reason
GIN-D-001	2026-10-08	The <code>grammarinnumbers</code> repository root <i>is</i> the independent <code>grammar-in-numbers/</code> workspace of the mandate. Mandate directories map as: <code>BOOK_CONTENT/</code> → <code>book/content/</code> , <code>SDK/</code> → <code>sdk/</code> , <code>EXPERIMENTS/</code> → <code>experiments/</code> , <code>WEB/</code> → <code>book/site/</code> + <code>book/tools/</code> , <code>TESTS/</code> → <code>sdk/tests/</code> + <code>book/tests/</code> , <code>BIBLIOGRAPHY/</code> → <code>book/content/bibliography.json</code> .	The repository was empty and dedicated to this publication; the CGT layout (<code>book/sdk/experiments</code>) is the family convention and keeps both publications navigable the same way.
GIN-D-002	2026-10-08	The CGT repository is read-only. It was read (ledger, book tooling) and never written; <code>git status</code> in the CGT checkout was verified clean at the end of the session. No CGT code is imported at runtime or build time.	Mandate §34.
GIN-D-003	2026-10-08	Primary experimental metric: counted steps under a stated model (rules, flips, gates/depth, 32-bit limb operations, division steps). Wall-clock only in <code>_s</code> fields.	Same discipline as CGT-D-003; mandatory bit complexity (mandate §26).
GIN-D-004	2026-10-08	SDK and experiments: Python standard library only. Machine experiments additionally use the host's gcc, clang, javac/java, rustc, go and node, with versions recorded.	Self-containment; the machine questions cannot be answered without real toolchains.
GIN-D-005	2026-10-08	Inverse operations are modelled as converse relations , and admissibility as	Sol
GIN-D-006	2026-10-08	A typing layer was added between syntax and semantics (four layers instead of CGT's three).	Literals such as 0.5 in $\mathbb{Z}$ and 2147483648 in <code>int32</code> fail neither syntactically nor semantically.
GIN-D-007	2026-10-08	The IEEE simulator computes exactly on rationals and rounds once. It is validated against the host's hardware results <i>and flags</i> , not only against Python floats.	Python cannot read flags and raises on <code>x/0</code> ; hardware is the only independent oracle available.
GIN-D-008	2026-10-08	After EXP-005's first run (101 flag mismatches, all signaling-NaN operands) the simulator was extended to model signaling NaNs, and the run repeated. The failed run is recorded (GIN-REJ-002).	Corrections preserve provenance.
GIN-D-009	2026-10-08	Exponents are evaluated in exact $\mathbb{Z}$ in every domain.	0^{-1} in $\mathbb{Z}/6$ had evaluated to 0 (GIN-NEG-013): exponentiation is an action of the integers, not a ring operation.
GIN-D-010	2026-10-08	EXP-012's integer-representability search is exhaustive only for $p \leq 11$; for binary32/64 it checks $1 \dots 2^{16}$ and a 4,096-wide window below 2^p .	A full search to 2^{24} with exact rationals did not finish in reasonable time; the window plus the proof (PROP-030) is sufficient and stated.
GIN-D-011	2026-10-08	EXP-011 lifts CPython's int-string conversion cap (<code>sys.set_int_max_str_digits(0)</code>) only to build reference values, and records the default cap.	The cap is itself an observation about cost (GIN-OBS-014).
GIN-D-012	2026-10-08	Euclid average-step data are reported against Porter's formula with the convention offset (-1) stated rather than "corrected away".	Different counting conventions; GIN-OBS-011.
GIN-D-013	2026-10-08	References are verified at metadata level by web search where possible and marked per entry (metadata-verified , secondary-only , unverified). An unverifiable "Foster 1947" was dropped in favour of Smullyan 1961 and Forslund 1995.	No invented citations.
GIN-D-014	2026-10-08	The book's build pipeline follows CGT's design (Markdown + KaTeX at build time, status-badged environments, build fails on any integrity error, executed examples checked at build time) but is a separate implementation in <code>book/tools/</code> . The visual identity shares CGT's typography and paper palette with a different accent (ultramarine instead of emerald).	Mandate §30, §34: same intellectual family, independent work.
GIN-D-015	2026-10-08	The website's laboratory runs a JavaScript port of the evaluator (<code>book/site/assets/js/gin-core.js</code>), and every lab result class is checked against fixtures generated from the Python SDK (<code>book/data/lab_fixtures.json</code>) by <code>book/tests</code> .	A lab that disagrees with the SDK would be a second, unchecked theory.

Bibliography

1. **Alford, Granville & Pomerance 1994.** W. R. Alford, A. Granville, C. Pomerance (1994). *There are infinitely many Carmichael numbers*. Annals of Mathematics 139:703–722. [metadata-verified]
2. **Arm ARM.** Arm Ltd. (2024). *Arm Architecture Reference Manual for A-profile architecture (DDI 0487), SDIV/UDIV*. Arm Ltd.. [secondary-only]
3. **Bergstra & Tucker 2007.** J. A. Bergstra, J. V. Tucker (2007). *The rational numbers as an abstract data type*. Journal of the ACM 54(2), Article 7. [metadata-verified]
4. **Blelloch 1990.** G. E. Blelloch (1990). *Prefix sums and their applications*. Technical report CMU-CS-90-190, Carnegie Mellon University. [metadata-verified]
5. **Blumensath & Grädel 2000.** A. Blumensath, E. Grädel (2000). *Automatic structures*. Proceedings of the 15th IEEE Symposium on Logic in Computer Science (LICS), 51–62. [metadata-verified]
6. **Brent & Kung 1982.** R. P. Brent, H. T. Kung (1982). *A regular layout for parallel adders*. IEEE Transactions on Computers C-31(3):260–264. [metadata-verified]
7. **Brent & Zimmermann 2010.** R. P. Brent, P. Zimmermann (2010). *Modern Computer Arithmetic*. Cambridge University Press. [unverified]
8. **Büchi 1960.** J. R. Büchi (1960). *Weak second-order arithmetic and finite automata*. Zeitschrift für mathematische Logik und Grundlagen der Mathematik 6:66–92. [metadata-verified]
9. **Burks, Goldstine & von Neumann 1946.** A. W. Burks, H. H. Goldstine, J. von Neumann (1946). *Preliminary discussion of the logical design of an electronic computing instrument*. Report to the U.S. Army Ordnance Department, Institute for Advanced Study. [secondary-only]
10. **Carlström 2004.** J. Carlström (2004). *Wheels — on division by zero*. Mathematical Structures in Computer Science 14(1):143–184. [metadata-verified]
11. **Charikar et al. 2005.** M. Charikar, E. Lehman, D. Liu, R. Panigrahy, M. Prabhakaran, A. Sahai, A. Shelat (2005). *The smallest grammar problem*. IEEE Transactions on Information Theory 51(7):2554–2576. [metadata-verified]
12. **Church 1936.** A. Church (1936). *An unsolvable problem of elementary number theory*. American Journal of Mathematics 58(2):345–363. [unverified]
13. **Church 1941.** A. Church (1941). *The Calculi of Lambda-Conversion*. Annals of Mathematics Studies 6, Princeton University Press. [unverified]
14. **Cobham 1969.** A. Cobham (1969). *On the base-dependence of sets of numbers recognizable by finite automata*. Mathematical Systems Theory 3:186–192. [metadata-verified]
15. **Cormen et al. 2009.** T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein (2009). *Introduction to Algorithms (3rd ed.)*. MIT Press. [unverified]
16. **CVE-2020-10735.** Python Software Foundation (2022). *Integer string conversion length limitation (CVE-2020-10735)*. Python documentation, stdtypes: int_max_str_digits. [secondary-only]
17. **Dedekind 1888.** R. Dedekind (1888). *Was sind und was sollen die Zahlen?*. Vieweg, Braunschweig. [unverified]
18. **Dixon 1970.** J. D. Dixon (1970). *The number of steps in the Euclidean algorithm*. Journal of Number Theory 2:414–422. [metadata-verified]
19. **ECMA-262.** Ecma International (2024). *ECMAScript Language Specification (Number::divide, BigInt::divide)*. Ecma International. [secondary-only]
20. **Euclid, Elements.** Euclid (c. 300 BCE). *Elements, Book VII, Propositions 1–2*. c. 300 BCE; e.g. T. L. Heath (trans.), Cambridge University Press 1908. [unverified]
21. **Forslund 1995.** R. R. Forslund (1995). *A logical alternative to the existing positional number system*. Southwest Journal of Pure and Applied Mathematics 1:27–29. [secondary-only]
22. **Go Spec.** The Go Authors (2024). *The Go Programming Language Specification: arithmetic operators, integer operators*. go.dev. [secondary-only]
23. **Gödel 1931.** K. Gödel (1931). *Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I*. Monatshefte für Mathematik und Physik 38:173–198. [unverified]
24. **Goldberg 1991.** D. Goldberg (1991). *What every computer scientist should know about floating-point arithmetic*. ACM Computing Surveys 23(1):5–48. [metadata-verified]
25. **Harvey & van der Hoeven 2021.** D. Harvey, J. van der Hoeven (2021). *Integer multiplication in time $O(n \log n)$* . Annals of Mathematics 193(2):563–617. [metadata-verified]
26. **Heilbronn 1969.** H. Heilbronn (1969). *On the average length of a class of finite continued fractions*. In Number Theory and Analysis (Papers in honor of Edmund Landau), Plenum, 87–96. [unverified]
27. **Hodgson 1983.** B. R. Hodgson (1983). *Décidabilité par automate fini*. Annales des Sciences Mathématiques du Québec 7(1):39–57. [metadata-verified]
28. **IEEE 754-2019.** IEEE Computer Society (2019). *IEEE Standard for Floating-Point Arithmetic (IEEE Std 754-2019)*. IEEE. [secondary-only]
29. **Intel SDM.** Intel Corporation (2024). *Intel 64 and IA-32 Architectures Software Developer's Manual, Vol. 2 (IDIV, DIV)*. Intel. [secondary-only]
30. **ISO C17.** ISO/IEC JTC1/SC22/WG14 (2018). *ISO/IEC 9899:2018, Programming languages — C (§6.5.5 multiplicative operators)*. ISO. [secondary-only]
31. **JLS SE 21.** J. Gosling et al. (2023). *The Java Language Specification, Java SE 21 Edition (§15.17.2)*. Oracle. [secondary-only]
32. **Kadaan (CGT) 2026.** A. Kadaan (research program); Claude (research assistant) (2026). *Computational Grammar Theory: Structure, Operation, and the Cost of Resolution — research ledger and working edition 0.1*. github.com/akadaan310/computationalgrammartheory (read-only reference, commit b3a29f3). [metadata-verified]

33. **Kahan 1987**. W. Kahan (1987). *Branch cuts for complex elementary functions, or much ado about nothing's sign bit*. In A. Iserles, M. J. D. Powell (eds.), *The State of the Art in Numerical Analysis*, Clarendon Press, 165–211. [metadata-verified]
34. **Karatsuba & Ofman 1962**. A. Karatsuba, Yu. Ofman (1962). *Multiplication of many-digital numbers by automatic computers*. Doklady Akademii Nauk SSSR 145:293–294. [unverified]
35. **Khoussainov & Nerode 1995**. B. Khoussainov, A. Nerode (1995). *Automatic presentations of structures*. Logic and Computational Complexity, LNCS 960, Springer, 367–392. [metadata-verified]
36. **Knuth 1978**. D. E. Knuth (1978). *The average time for carry propagation*. Indagationes Mathematicae (Proc. KNAW) 40:238–242. [metadata-verified]
37. **Knuth 1997**. D. E. Knuth (1997). *The Art of Computer Programming, Vol. 2: Seminumerical Algorithms (3rd ed.)*. Addison-Wesley. [unverified]
38. **Kogge & Stone 1973**. P. M. Kogge, H. S. Stone (1973). *A parallel algorithm for the efficient solution of a general class of recurrence equations*. IEEE Transactions on Computers C-22(8):786–793. [metadata-verified]
39. **Korselt 1899**. A. Korselt (1899). *Problème chinois*. L'Intermédiaire des Mathématiciens 6:142–143. [metadata-verified]
40. **Ladner & Fischer 1980**. R. E. Ladner, M. J. Fischer (1980). *Parallel prefix computation*. Journal of the ACM 27(4):831–838. [metadata-verified]
41. **Lamé 1844**. G. Lamé (1844). *Note sur la limite du nombre des divisions dans la recherche du plus grand commun diviseur entre deux nombres entiers*. Comptes rendus de l'Académie des sciences 19:867–870. [unverified]
42. **mathlib Community 2020**. The mathlib Community (2020). *The Lean mathematical library*. Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020), 367–381. [unverified]
43. **Miller 1976**. G. L. Miller (1976). *Riemann's hypothesis and tests for primality*. Journal of Computer and System Sciences 13(3):300–317. [metadata-verified]
44. **O'Connor & Robertson (MacTutor)**. J. J. O'Connor, E. F. Robertson (2000). *Brahmagupta (biography)*. MacTutor History of Mathematics Archive, University of St Andrews. [metadata-verified]
45. **Peano 1889**. G. Peano (1889). *Arithmetices principia, nova methodo exposita*. Bocca, Turin. [unverified]
46. **Plofker 2009**. K. Plofker (2009). *Mathematics in India*. Princeton University Press. [unverified]
47. **Pollard 1975**. J. M. Pollard (1975). *A Monte Carlo method for factorization*. BIT 15:331–334. [metadata-verified]
48. **Pomerance, Selfridge & Wagstaff 1980**. C. Pomerance, J. L. Selfridge, S. S. Wagstaff Jr. (1980). *The pseudoprimes to $25 \cdot 10^9$* . Mathematics of Computation 35:1003–1026. [metadata-verified]
49. **Porter 1975**. J. W. Porter (1975). *On a theorem of Heilbronn*. Mathematika 22:20–28. [unverified]
50. **Python Reference**. Python Software Foundation (2024). *The Python Language Reference, §6.7 Binary arithmetic operations*. python.org. [secondary-only]
51. **Rabin 1980**. M. O. Rabin (1980). *Probabilistic algorithm for testing primality*. Journal of Number Theory 12:128–138. [metadata-verified]
52. **RISC-V ISA Manual**. RISC-V International (2019). *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA — M extension*. RISC-V International (ratified 20191213; later versions keep the division semantics). [metadata-verified]
53. **Rust Reference**. The Rust Project (2024). *The Rust Reference: arithmetic and logical binary operators; integer overflow*. rust-lang.org. [secondary-only]
54. **Sklansky 1960**. J. Sklansky (1960). *Conditional-sum addition logic*. IRE Transactions on Electronic Computers EC-9:226–231. [metadata-verified]
55. **Smullyan 1961**. R. M. Smullyan (1961). *Theory of Formal Systems*. Annals of Mathematics Studies 47, Princeton University Press. [metadata-verified]
56. **Sorenson & Webster 2017**. J. Sorenson, J. Webster (2017). *Strong pseudoprimes to twelve prime bases*. Mathematics of Computation 86:985–1003. [metadata-verified]
57. **Stein 1967**. J. Stein (1967). *Computational problems associated with Racah algebra*. Journal of Computational Physics 1(3):397–405. [metadata-verified]